Héctor Pons, Daniel Benito, Mateo Kristich,
Oscar Almeida, Paz Valdés

# Práctica 2: Estudio del comportamiento de protocolos mediante el análisis de paquetes con Wireshark

**Autores:**
**Miguel García Pineda**
**Paloma Moreno Clarí**
**Juan Manuel Orduña Huertas**

***IMPORTANTE**: El alumno deberá haber leído el guion de prácticas con anterioridad a su sesión, ya que al inicio de la misma se le realizará un examen. Además, deberá contestar las preguntas planteadas y anotar en las zonas reservadas al efecto la respuesta obtenida, pues ello facilitará tanto la corrección y visualización por parte del profesor, como su posterior estudio.*

## Objetivo y descripción general

**Objetivo**:

El objetivo de esta práctica es familiarizarse con la una de las herramientas de análisis de tráfico en red más utilizada en el ámbito académico: **Wireshark**, estudiando el comportamiento de los algunos de protocolos de transporte y aplicación más utilizados: TCP, UDP, DNS y HTTP.

**Trabajo Previo**:

Con anterioridad a la sesión de laboratorio, el alumno deberá haber leído el guion de prácticas donde se desarrollan todos los pasos a realizar, y haber repasado la teoría del funcionamiento de los protocolos objeto de estudio de esta sesión: TCP, UDP, DNS y HTTP. Disponible junto al enunciado de la práctica hay 2 anexos con algunos de los filtros de captura y visualización más utilizados en Wireshark.

**Al iniciar la sesión se le realizará un pequeño examen individual acerca de la memoria para comprobar su lectura y entendimiento. Dicho examen será el 30% de la nota la sesión, el resto dependerá de las respuestas realizadas a las preguntas que se realizan en la memoria.**

## Desarrollo de la práctica.

### *0.- Inicio*

Esta práctica se realizará en la partición de Linux-Redes entrando como usuario root y password admin-redes. El analizador de tramas Wireshark funciona igual sobre el sistema operativo Linux, Windows y MacOS y los comandos que utilizaremos por línea de comandos son muy semejantes.

Cuando necesitemos utilizar comandos que nos den información sobre la configuración de red del ordenador abriremos un **terminal**.

## *1.- Introducción: Wireshark y pruebas iniciales sobre protocolos de red*

**Wireshark** es el analizador de protocolos más popular en la actualidad. Se trata de un software de dominio público que está desarrollado para múltiples plataformas de sistemas operativos y múltiples interfaces físicas. Es capaz de decodificar más de 400 protocolos diferentes, desde el nivel de enlace al nivel de aplicación. El software puede obtenerse de forma gratuita de **www.wireshark.org**. Este programa es una evolución del conocido como *Ethereal*, pero su principal desarrollador pasó a trabajar para otra empresa y por problemas de registro de marcas tuvo que cambiarle el nombre.

Wireshark tiene una interfaz gráfica como se muestra en la Figura 1. Cuando se utiliza sobre una interfaz Ethernet puede funcionar en modo promiscuo y capturar todas las tramas de la red, o bien en modo no promiscuo y capturar solo las tramas dirigidas a su interfaz.

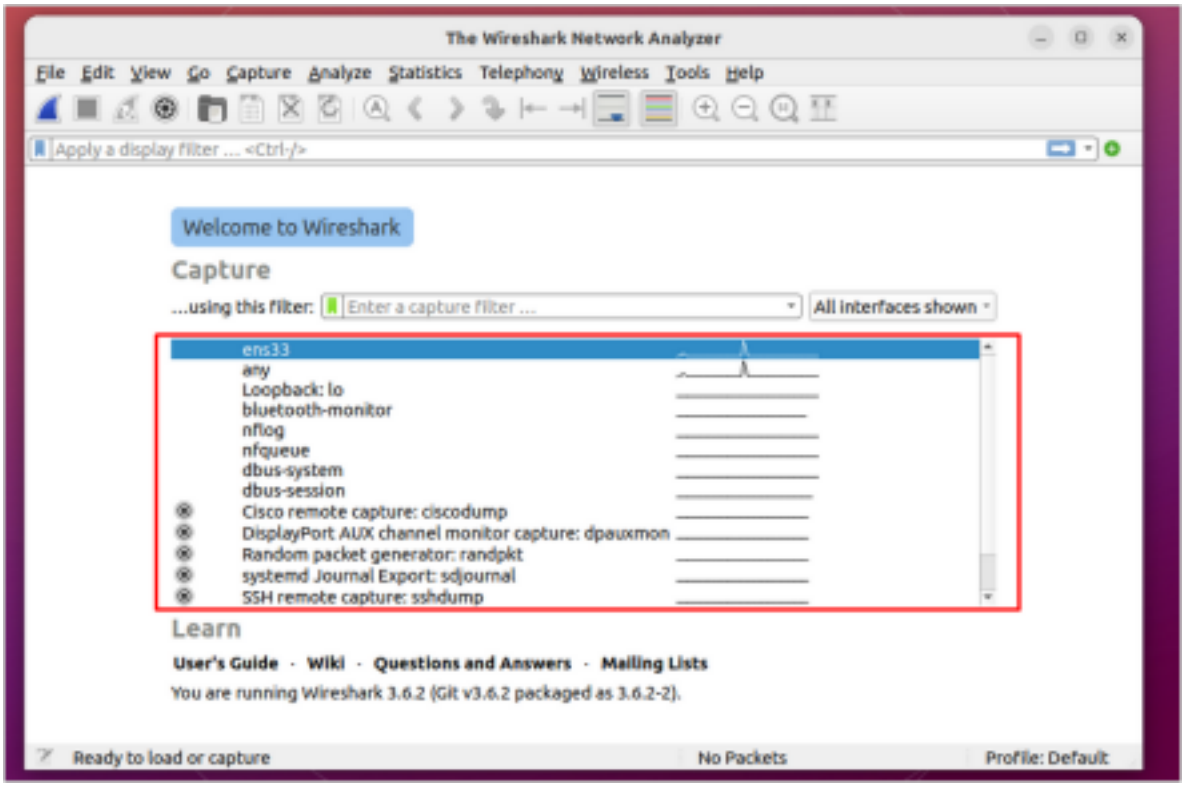



*Figura 1. Interfaz gráfica del analizador de protocolos Wireshark.*

En la lista de interfaces, como se observa en la Figura 1, se nos muestra la lista de interfaces disponibles, así como la actividad de cada interfazo. Normalmente no tendremos problemas en identificar la interfaz Ethernet, ya que recibe el nombre **enp2s0** en la distribución que está instalada en el laboratorio, aunque puede tener otro nombre. Si deseamos introducir un filtro de captura para la interfaz seleccionada podemos hacerlo. Una sugerencia es abrir una terminal y usar el comando **ip add** para conocer el número <ip> de la tarjeta de red del computador del laboratorio, y en el filtro de captura introducir el filtro **host <ip>.** Si pulsamos el botón con la cola azul de tiburón habiendo seleccionado la

interfaz Ethernet **enp2s0** empìeza de forma inmediata la captura.

*eccionada*



Cuando comienza la captura comienzan a aparecer las tramas capturadas en la ventana de la aplicación wireshark. La ventana de captura se divide en tres zonas como se muestra en la Figura 4. Puede que estas ventanas estén ocultas y haya que desplazar los separadores para verlas. En la superior se muestra la lista de tramas/paquetes capturados, a razón de una línea por paquete. En la intermedia se muestra en detalle la estructura del paquete seleccionado en la primera, y en la inferior se muestra el contenido del paquete en hexadecimal. Para analizar un paquete basta con pincharlo en la zona superior; entonces se puede seleccionar alguno de los campos que aparecen en la zona intermedia y desglosarlo en sus partes si las tiene (por ejemplo ver en detalle los campos de la cabecera IP). Al seleccionar algún campo de la zona intermedia automáticamente queda seleccionado en la inferior el contenido en hexadecimal correspondiente, con lo que es posible analizar el contenido con todo detalle. En el ejemplo mostrado en la Figura 4 se ha seleccionado en la zona superior la trama número 13; en la zona intermedia se ha seleccionado el campo Type de la cabecera Ethernet y como puede verse en la zona inferior han quedado seleccionados automáticamente los bytes 13 y 14 de la trama, que en este caso tienen el valor hexadecimal 0800, que corresponde al protocolo IP.

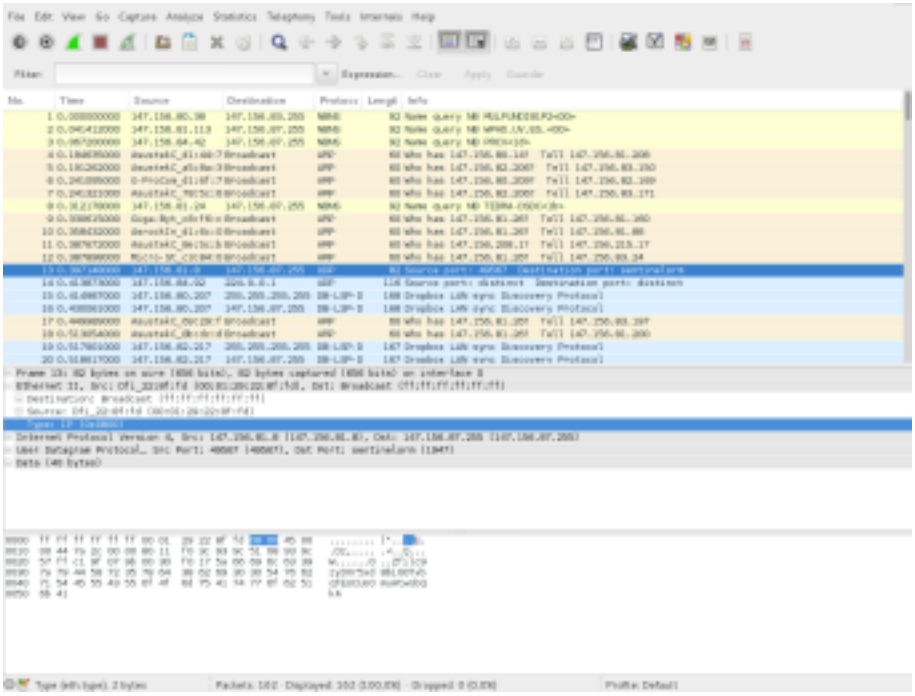


*Figura 4. Pantalla de capturas de Wireshark*

El analizador interpreta los protocolos desde el nivel de enlace al nivel de aplicación. La lista de protocolos que abarca para cada uno de los niveles es muy completa. Es interesante en este punto que le echéis un vistazo y vayáis pinchando por las diferentes zonas y ver, por ejemplo contenidos como los de las cabeceras de la trama Ethernet o el de las cabeceras del protocolo IP y/o IPv6.

Como la captura suele recoger una gran cantidad de paquetes, el analizador posee la opción



de poder crear filtros[1](*capture-filters*) que se aplican en el momento de hacer las capturas descartando aquellas tramas que no cumplan unas determinadas condiciones. Dichos filtros se crean desde el menú ***Capture*** y su submenú ***Options…*** escribiendo el filtro deseado dentro de la opción ***Capture Filter:***. Para poder modificar estos filtros debemos tener el proceso de capturas parado.

También existen los filtros de visualización. Estos filtros nos sirven para analizar el tráfico capturado, y son diferentes a los filtros de captura, en este caso, los filtros de visualización son más amplios y potentes, incluso son más fáciles de aplicar, y son de mucha utilidad para analizar el tráfico capturado. La principal diferencia frente a los filtros de captura es que con los filtros de captura solo capturamos el tráfico que corresponde con el filtro y en los filtros de visualización se captura todo el tráfico, pero sólo se visualiza aquel que coincida con el filtro aplicado. La introducción se realiza en la pantalla de captura en la parte superior donde se indica **Filter:**, (ver Figura 4). Para más información acerca de los filtros de visualización en Wireshark consultar el apéndice B.

### *2.- Estudio del protocolo de Transporte TCP utilizando Wireshark*

A continuación, estudiaremos del protocolo TCP mediante el análisis de los segmentos TCP enviados y recibidos en la transferencia de un archivo de 150 KB desde el ordenador a un servidor remoto. Comprobaremos el uso de los siguientes mecanismos en TCP:

- Números de secuencia y reconocimientos para conseguir la transferencia de datos fiable.
- Algoritmo de control de congestión de TCP.
- Mecanismo de control de flujo.
- También investigaremos el rendimiento de la conexión TCP entre el ordenador y el servidor.

### 2.1. Captura de una Transmisión TCP de un ordenador a un servidor remoto.

Previo a la exploración del comportamiento del protocolo TCP es necesario utilizar Wireshark para para realizar las capturas de paquetes intercambiados en la transferencia TCP de un archivo desde el ordenador a un servidor remoto.

Esta transferencia la realizaremos accediendo a una página web que permita subir el archivo de texto ASCII (disponible en el Aula Virtual, junto al guión de la práctica) a un servidor Web mediante el método HTTP POST. Antes de realizar esta transferencia habremos puesto ya el Wireshark en modo captura para analizar los segmentos TCP enviados y recibidos desde el ordenador.

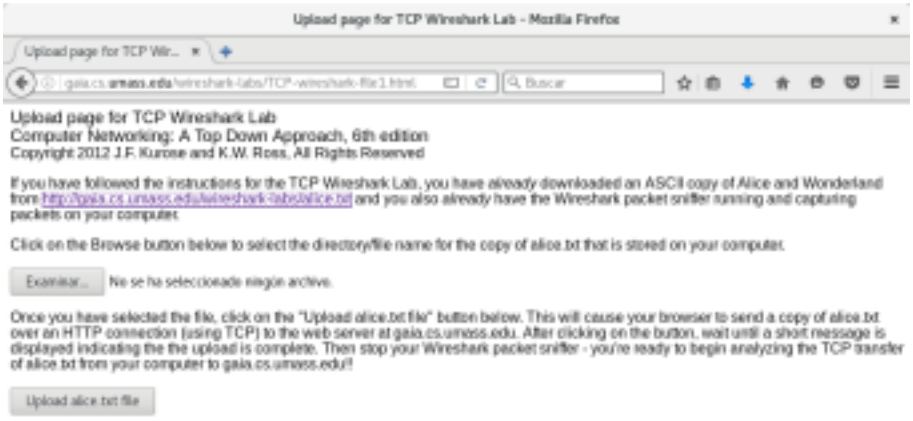
Pasos:

1. Guarda el archivo ASCII proporcionado en el equipo local.
2. Accede a http://gaia.cs.umass.edu/wireshark-labs/TCP-wireshark-file1.html. La

1 En el apéndice A y B se encuentra una explicación de los filtros y su sintaxis.



pantalla a la que se accede es semejante a la que se observa en la Figura




5.

*Figura 5. Web para analizar el comportamiento de TCP.*

3. Con el botón Browse introduce el nombre del archivo (ruta completa) que te habrás descargado en el equipo y que contiene texto ASCII. (ATENCIÓN: Antes de subirlo es necesario poner el Whireshark en modo Captura)
4. A continuación, pon en marcha Wireshark y comienza la captura de paquetes (*Capture-> Start*) y pulsa OK en la pantalla de opciones de captura de paquetes de Wireshark (*Capture Options)* (en este apartado no es necesario seleccionar opciones).
5. Regresa al navegador, pulsa el botón "*Upload* alice.txt *file*" para cargar el archivo al servidor gaia.cs.umass.edu. Una vez que se carga el archivo, se muestra en la ventana del navegador un mensaje de éxito en la subida.
6. Detén (*stop*) la captura de paquetes en Whireshark. La ventana de Wireshark será similar a la que se muestra a continuación.

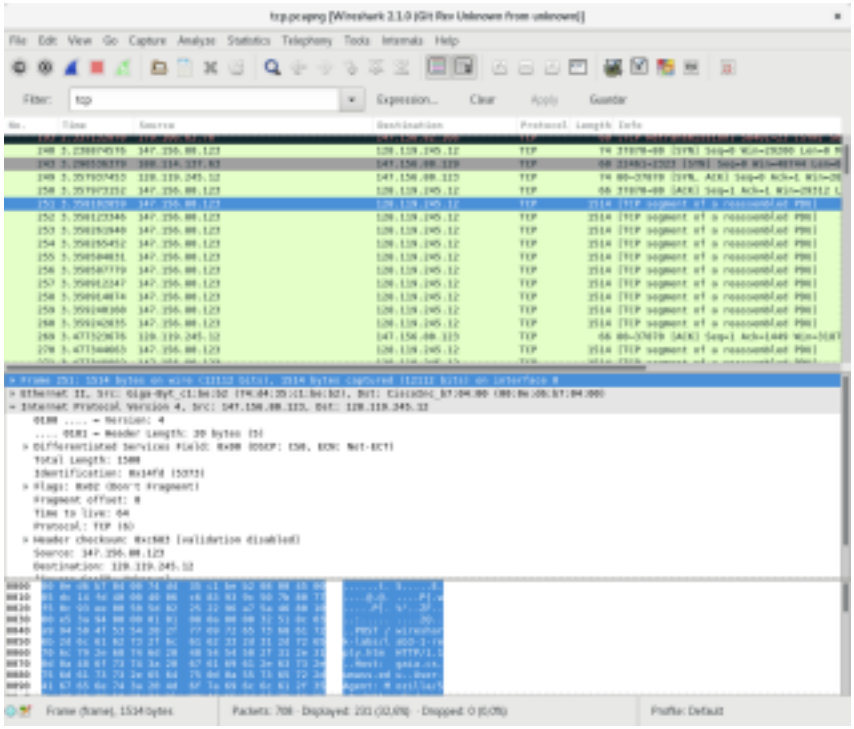




*Figura 4. Ejemplo de captura TCP con Wireshark.*

**2.2. Estudio inicial de la captura realizada.**

Para ello, en primer lugar, debemos filtrar los paquetes que aparecen en la ventana Wireshark mediante la introducción de "tcp" (en minúsculas, sin comillas, y presionando enter después de entrar) en la ventana de especificación de filtros que aparece en la parte superior de la ventana de Wireshark.

En la ventana de captura aparecerán una serie de mensajes TCP y HTTP entre el ordenador y el servidor gaia.cs.umass.edu, entre ellos el mensaje inicial del saludo a tres vías que contiene un mensaje SYN; segmentos de TCP ACK desde el servidor al ordenador; y el mensaje HTTP POST. Cuando un único mensaje HTTP se envía en varios segmento TCP, aparecen mensajes en la columna de información de Whireshark para indicarlo (una explicación más detallada puedes encontrarla en el apartado 4 de este guión).

Responde a las siguientes preguntas, utilizando la información proporcionada sobre la captura realizada por Whireshark:

1. ¿Cuál es la dirección IP y número de puerto TCP utilizado por tu equipo para transferir el archivo al servidor? (Para responder a esta pregunta, puedes seleccionar un mensaje HTTP y explorar los detalles del paquete TCP utilizado para transportarlo, usando los "*details of the selected packet header window*".

| | |
|---|---|
| **Respuesta:** | **IP utilizada por el equipo:** 192.168.0.42<br>**Puerto TCP del cliente:** 58543 |

**2.** ¿Cuál es la dirección IP del servidor? Qué número de puerto se está utilizando para enviar y recibir segmentos TCP en esta conexión?

| | |
|---|---|
| **Respuesta:** | **IP del servidor:** 128.119.245.12<br>**Puerto del servidor:** 80 |

Puesto que en este apartado estamos analizando el comportamiento del protocolo TCP (HTTP lo veremos en un apartado posterior), cambiaremos la ventana de Whireshark en que se muestra la "lista de paquetes capturados" para que muestre información sobre los segmentos TCP que contienen los mensajes HTTP, en lugar de sobre los mensajes HTTP. Para ello selecciona *Analyze>Enabled Protocols*. A continuación, desactiva la casilla HTTP y selecciona *OK*. La ventana que aparece es semejante a:



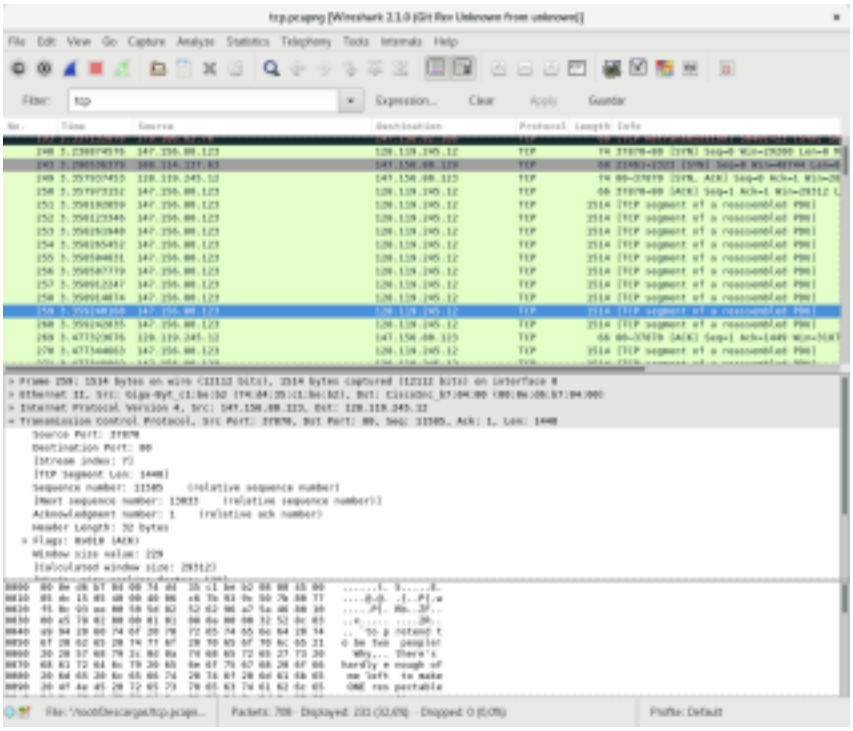


*Figura 6. Ejemplo de captura TCP sin HTTP con Wireshark.*

Esta es la información que usaremos para estudiar el comportamiento de TCP en los puntos 2.3, y 2.4 de la práctica.

### 2.3. Conceptos y funcionamiento básico de TCP

Responde a las siguientes preguntas para los segmentos TCP:

**3.** ¿Cuál es el número de secuencia del segmento TCP SYN que se utiliza para iniciar la conexión TCP entre el cliente y gaia.cs.umass.edu? ¿Qué característica identifica este segmento como un segmento SYN?

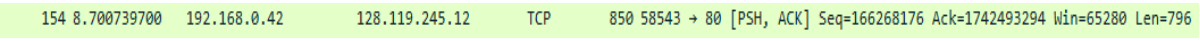
| | |
|---|---|
| **Respuesta:** | El número de secuencia del SYN es 0 en relativo (raw 166268175). Lo identifica como SYN el flag SYN = 0 con ACK = 0 |

**4.** ¿Cuál es el número de secuencia del segmento SYNACK enviado por gaia.cs.umass.edu al cliente en respuesta al SYN? ¿Cuál es el valor del campo de acuse de recibo en el segmento SYNACK? ¿Cómo gaia.cs.umass.edu determinar ese valor? ¿Qué característica identifica este segmento como un segmento SYNACK?



| | |
|---|---|
| **Respuesta:** | El seq del SYN-ACK es 0 en relativo (raw 1742493293) y su campo de acuse es 1 en relativo (raw 166268176). El servidor lo calcula como el número de secuencia del SYN del cliente + 1. Lo identifica como SYN-ACK que tiene SYN=1 y ACK=1 a la vez. |

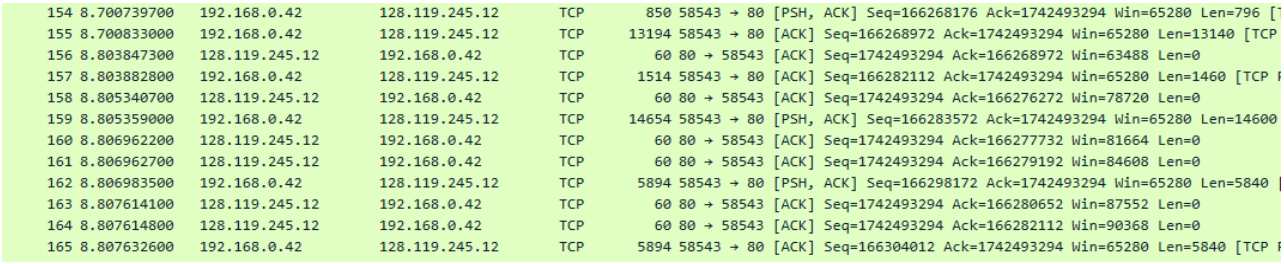
**5.** ¿Cuál es el número de secuencia del segmento TCP que contiene el comando HTTP POST? Ten en cuenta que con el fin de encontrar el comando POST, deberás explorar el campo de contenido del paquete en la parte inferior de la ventana Wireshark, en busca de un segmento con un "POST" dentro de su campo de datos.




| | |
|---|---|
| **Respuesta:** | Es el paquete 154, seq relativo 1, con 796 bytes de datos.<br>Dentro está POST /wireshark-labs/lab3-1-reply.htm HTTP/1.1. Es el primer segmento de datos de conexión |

**6.** Considera el segmento TCP que contiene el HTTP POST como el primer segmento de la conexión TCP. ¿Cuáles son los números de secuencia de los primeros seis segmentos en la conexión TCP (incluyendo el segmento que contiene el HTTP POST)? ¿En qué momento se envió a cada segmento? ¿Cuándo se ha recibido el

ACK para cada segmento?
Teniendo en cuenta la diferencia entre el tiempo de envío de cada segmento TCP, y la recepción de su reconocimiento, ¿cuál es el valor de RTT para cada uno de los seis segmentos?

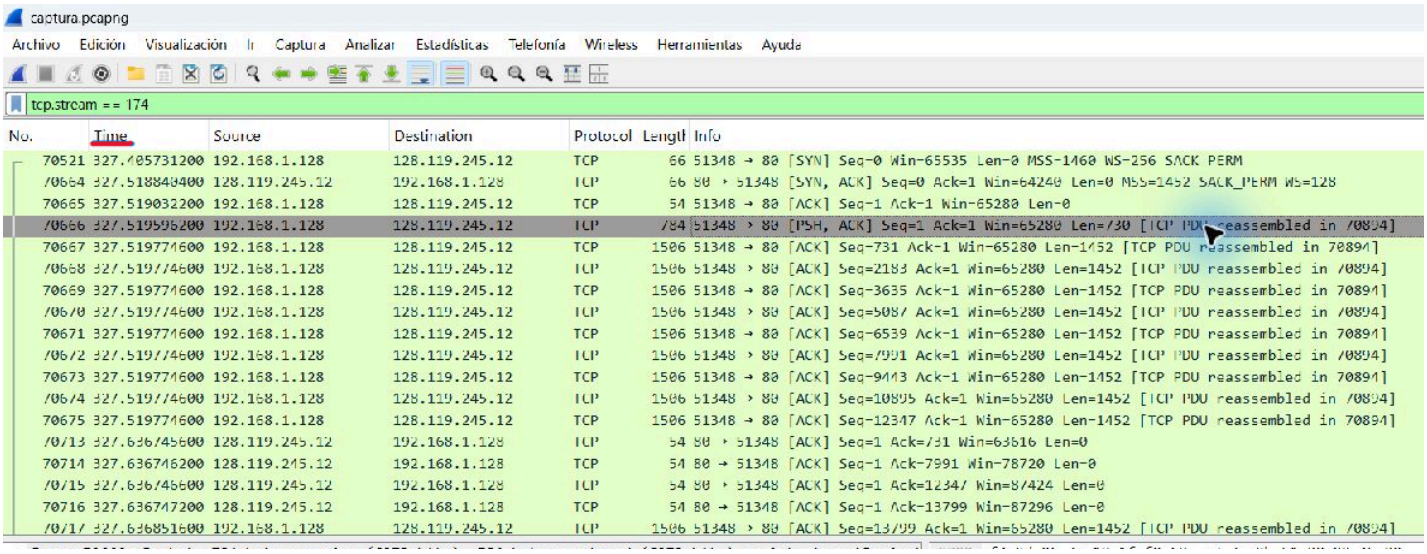


```
154 8.700739700   192.168.0.42      128.119.245.12    TCP      850 58543 → 80 [PSH, ACK] Seq=166268176 Ack=1742493294 Win=65280 Len=796 [
155 8.700833000   192.168.0.42      128.119.245.12    TCP    13194 58543 → 80 [ACK] Seq=166268972 Ack=1742493294 Win=65280 Len=13140 [TCP
156 8.803847300   128.119.245.12    192.168.0.42      TCP       60 80 → 58543 [ACK] Seq=1742493294 Ack=166268972 Win=63488 Len=0
157 8.803882800   192.168.0.42      128.119.245.12    TCP     1514 58543 → 80 [ACK] Seq=166282112 Ack=1742493294 Win=65280 Len=1460 [TCP
158 8.805340700   128.119.245.12    192.168.0.42      TCP       60 80 → 58543 [ACK] Seq=1742493294 Ack=166276272 Win=78720 Len=0
159 8.805359000   192.168.0.42      128.119.245.12    TCP    14654 58543 → 80 [PSH, ACK] Seq=166283572 Ack=1742493294 Win=65280 Len=14600
160 8.806962200   128.119.245.12    192.168.0.42      TCP       60 80 → 58543 [ACK] Seq=1742493294 Ack=166277732 Win=81664 Len=0
161 8.806962700   128.119.245.12    192.168.0.42      TCP       60 80 → 58543 [ACK] Seq=1742493294 Ack=166279192 Win=84608 Len=0
162 8.806983500   192.168.0.42      128.119.245.12    TCP     5894 58543 → 80 [PSH, ACK] Seq=166298172 Ack=1742493294 Win=65280 Len=5840
163 8.807614100   128.119.245.12    192.168.0.42      TCP       60 80 → 58543 [ACK] Seq=1742493294 Ack=166280652 Win=87552 Len=0
164 8.807614800   128.119.245.12    192.168.0.42      TCP       60 80 → 58543 [ACK] Seq=1742493294 Ack=166282112 Win=90368 Len=0
165 8.807632600   192.168.0.42      128.119.245.12    TCP     5894 58543 → 80 [ACK] Seq=166304012 Ack=1742493294 Win=65280 Len=5840 [TCP
```

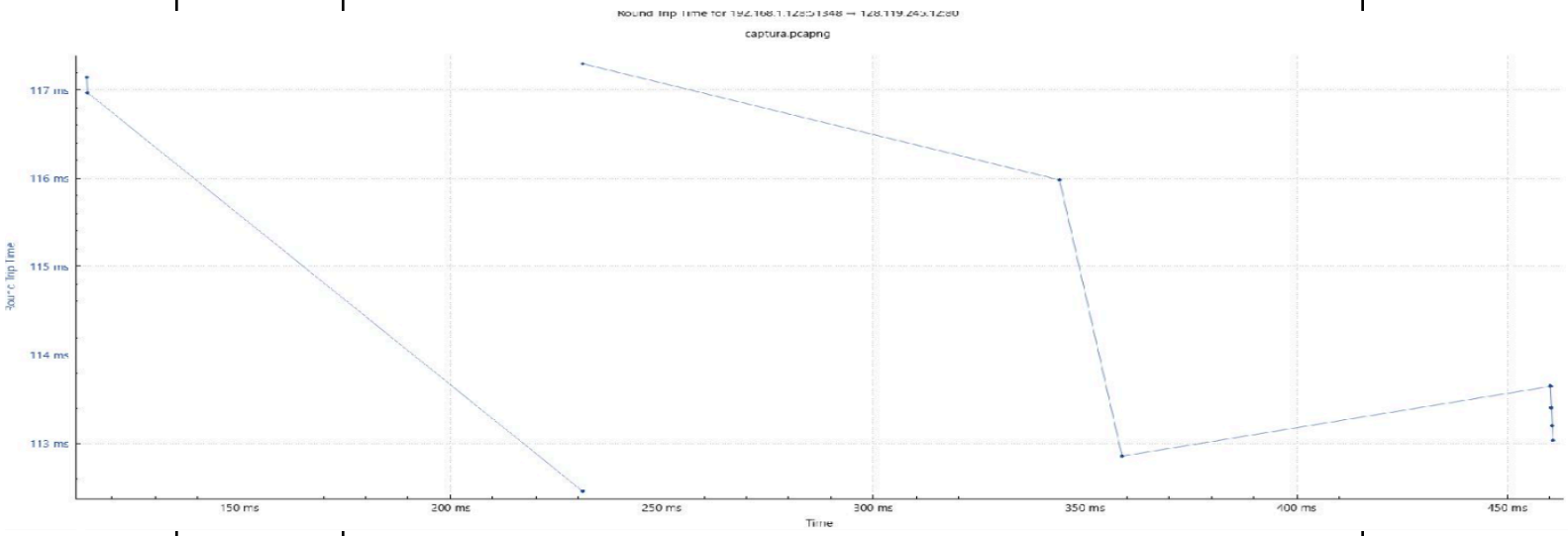
**Respuesta:**

| **Seg** | **Pkt** | **Seq** | **Bytes** | **Envío (s)** | **ACK (s)** | **RTT (s)** |
|---|---|---|---|---|---|---|
| 1 (POST) | 154 | 1 | 796 | 8.700740 | 8.803847 | 0.103108 |
| 2 | 155 | 797 | 13140 | 8.700833 | 8.807615 | 0.106782 |
| 3 | 157 | 13937 | 1460 | 8.803883 | 8.908359 | 0.104476 |
| 4 | 159 | 15397 | 14600 | 8.805359 | 8.911376 | 0.106017 |
| 5 | 162 | 29997 | 5840 | 8.806983 | 8.915290 | 0.108306 |
| 6 | 165 | 35837 | 5840 | 8.807632 | 8.915291 | 0.107658 |

7. ¿Cuál es el valor estimado del RTT, *EstimatedRTT* después de la recepción de cada ACK? Para ello, supón que el valor estimado de RTT, *EstimatedRTT,* es igual al RTT medido para el primer segmento y, a continuación, se calcula utilizando la ecuación para el *EstimatedRTT*, para todos los segmentos posteriores.




*EstimatedRTT(K)* = α·*EstimatedRTT(K-1)* + *(1*-α)*RTT(K)* α=7/8




| | |
|---|---|
| **Respuesta:** | **De los seis primeros segmentos se reciben dos ACKs relevantes. El ACK del frame 70713 confirma el primer segmento y tiene una muestra RTT de 117,149 ms. Tomamos esa primera muestra como el primer EstimatedRTT. El ACK del frame 70714 confirma acumulativamente los segmentos 70667–70671, que se enviaron juntos; su muestra RTT es 116,972 ms. Como se recibió un solo ACK acumulativo, actualizamos EstimatedRTT una sola vez para ese ACK:**<br>**EstimatedRTT_2 = (7/8) * 117,149 + (1/8) * 116,972 = 117,127 ms**<br>**Por tanto, después del ACK 70713, EstimatedRTT es 117,149 ms; después del ACK 70714, es 117,127 ms.** |

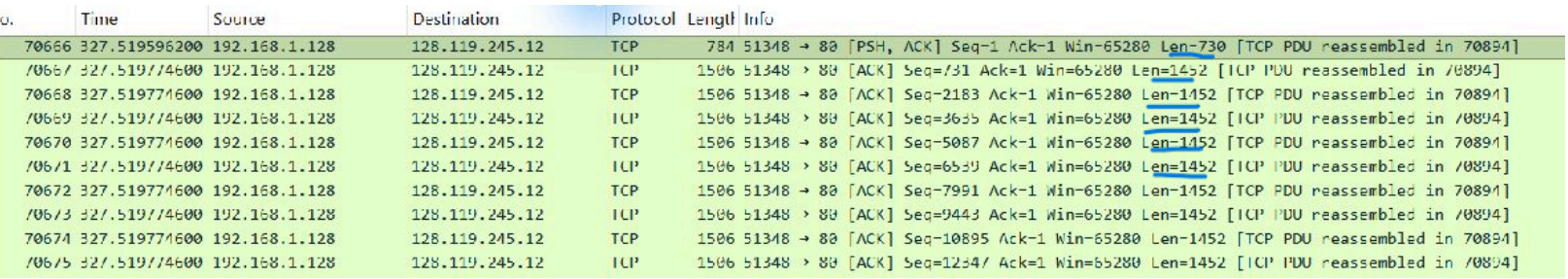


*Nota*: Wireshark también dispone de una herramienta que permite visualizar gráficamente

el RTT para cada uno de los segmentos TCP enviados. Para ello, selecciona un segmento TCP de los paquetes capturados que tenga como origen el cliente y como destino el servidor, en la ventana "*listing of captured packets*" ventana que se envía desde el cliente al servidor. A continuación, selecciona: *Statistics-> TCP Stream Graph-> Round Trip Time Graph.* Para ello debes seleccionar el stream y la dirección adecuado al flujo analizado.

8. ¿Cuál es la longitud de cada uno de los seis primeros segmentos TCP?

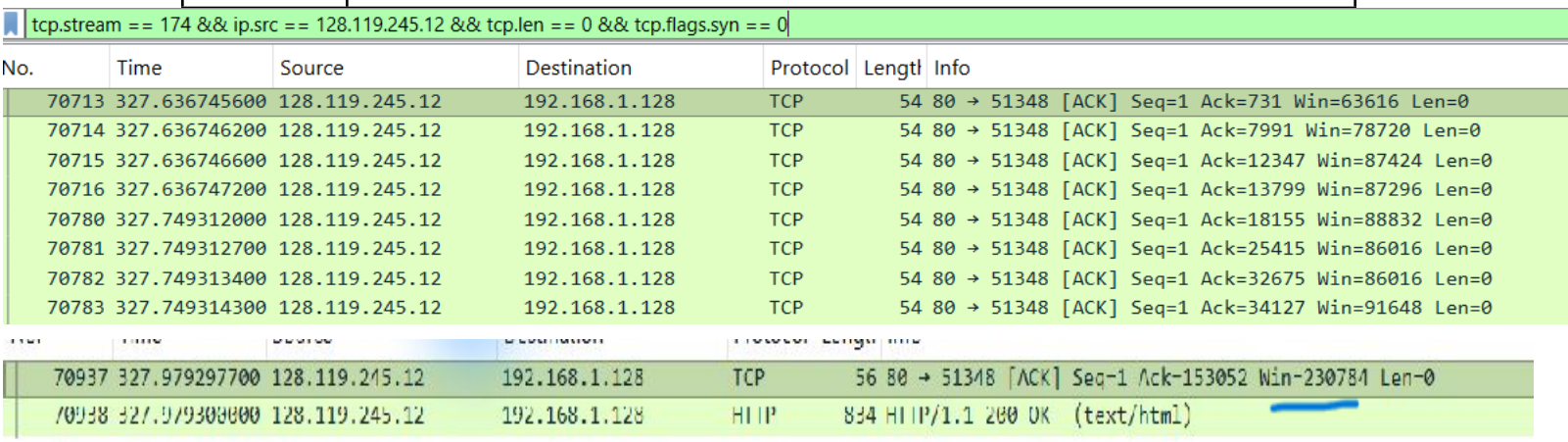
| | |
|---|---|
| **Respuesta :** | **Son 730, 1452, 1452, 1452, 1452 y 1452 bytes.** |



| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 70666 | 327.519596200 | 192.168.1.128 | 128.119.245.12 | TCP | 784 | 51348 → 80 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=730 [TCP PDU reassembled in 70894] |
| 70667 | 327.519774600 | 192.168.1.128 | 128.119.245.12 | TCP | 1506 | 51348 → 80 [ACK] Seq=731 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894] |
| 70668 | 327.519774600 | 192.168.1.128 | 128.119.245.12 | TCP | 1506 | 51348 → 80 [ACK] Seq=2183 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894] |
| 70669 | 327.519774600 | 192.168.1.128 | 128.119.245.12 | TCP | 1506 | 51348 → 80 [ACK] Seq=3635 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894] |
| 70670 | 327.519774600 | 192.168.1.128 | 128.119.245.12 | TCP | 1506 | 51348 → 80 [ACK] Seq=5087 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894] |
| 70671 | 327.519774600 | 192.168.1.128 | 128.119.245.12 | TCP | 1506 | 51348 → 80 [ACK] Seq=6539 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894] |
| 70672 | 327.519774600 | 192.168.1.128 | 128.119.245.12 | TCP | 1506 | 51348 → 80 [ACK] Seq=7991 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894] |
| 70673 | 327.519774600 | 192.168.1.128 | 128.119.245.12 | TCP | 1506 | 51348 → 80 [ACK] Seq=9443 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894] |
| 70674 | 327.519774600 | 192.168.1.128 | 128.119.245.12 | TCP | 1506 | 51348 → 80 [ACK] Seq=10895 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894] |
| 70675 | 327.519774600 | 192.168.1.128 | 128.119.245.12 | TCP | 1506 | 51348 → 80 [ACK] Seq=12347 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894] |

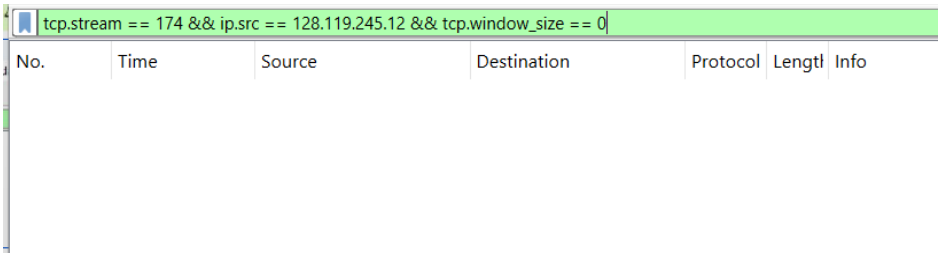
9. ¿Cuál es mínimo espacio (aconsejable) disponible en el buffer para la recepción de la traza o captura completa? ¿La falta de espacio en el buffer del receptor puede tener en algún caso algún efecto sobre el servidor?

| | |
|---|---|
| **Respuesta :** | **El mínimo anunciado por el servidor es 63.616 bytes ( 62,1 KiB), en el paquete 70713: ventana 497 × factor de escala 128. Después crece hasta 230.784 bytes. Nunca llega a 0. Los bytes en vuelo no pasaron de 62.297, así que ni siquiera alcanzaron esa ventana mínima. La falta de buffer en el receptor no frenó al emisor (el cliente) en esta transferencia. Si la ventana llegara a 0, el cliente pararía hasta recibir una actualización de ventana (control de flujo).** |



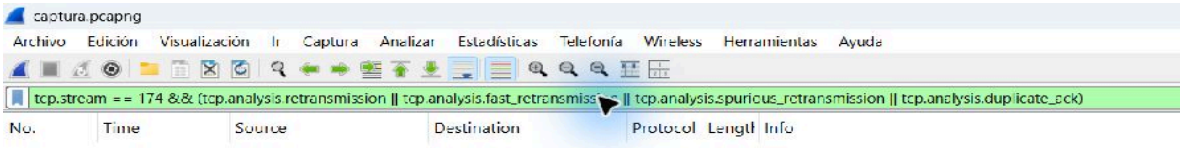
tcp.stream == 174 && ip.src == 128.119.245.12 && tcp.len == 0 && tcp.flags.syn == 0

| No. | Time | Source | Destination | Protocol | Length | Info |
|---|---|---|---|---|---|---|
| 70713 | 327.636745600 | 128.119.245.12 | 192.168.1.128 | TCP | 54 | 80 → 51348 [ACK] Seq=1 Ack=731 Win=63616 Len=0 |
| 70714 | 327.636746200 | 128.119.245.12 | 192.168.1.128 | TCP | 54 | 80 → 51348 [ACK] Seq=1 Ack=7991 Win=78720 Len=0 |
| 70715 | 327.636746600 | 128.119.245.12 | 192.168.1.128 | TCP | 54 | 80 → 51348 [ACK] Seq=1 Ack=12347 Win=87424 Len=0 |
| 70716 | 327.636747200 | 128.119.245.12 | 192.168.1.128 | TCP | 54 | 80 → 51348 [ACK] Seq=1 Ack=13799 Win=87296 Len=0 |
| 70780 | 327.749312000 | 128.119.245.12 | 192.168.1.128 | TCP | 54 | 80 → 51348 [ACK] Seq=1 Ack=18155 Win=88832 Len=0 |
| 70781 | 327.749312700 | 128.119.245.12 | 192.168.1.128 | TCP | 54 | 80 → 51348 [ACK] Seq=1 Ack=25415 Win=86016 Len=0 |
| 70782 | 327.749313400 | 128.119.245.12 | 192.168.1.128 | TCP | 54 | 80 → 51348 [ACK] Seq=1 Ack=32675 Win=86016 Len=0 |
| 70783 | 327.749314300 | 128.119.245.12 | 192.168.1.128 | TCP | 54 | 80 → 51348 [ACK] Seq=1 Ack=34127 Win=91648 Len=0 |
| 70937 | 327.979297700 | 128.119.245.12 | 192.168.1.128 | TCP | 56 | 80 → 51348 [ACK] Seq=1 Ack=153052 Win=230784 Len=0 |
| 70938 | 327.979308800 | 128.119.245.12 | 192.168.1.128 | HTTP | 834 | HTTP/1.1 200 OK (text/html) |

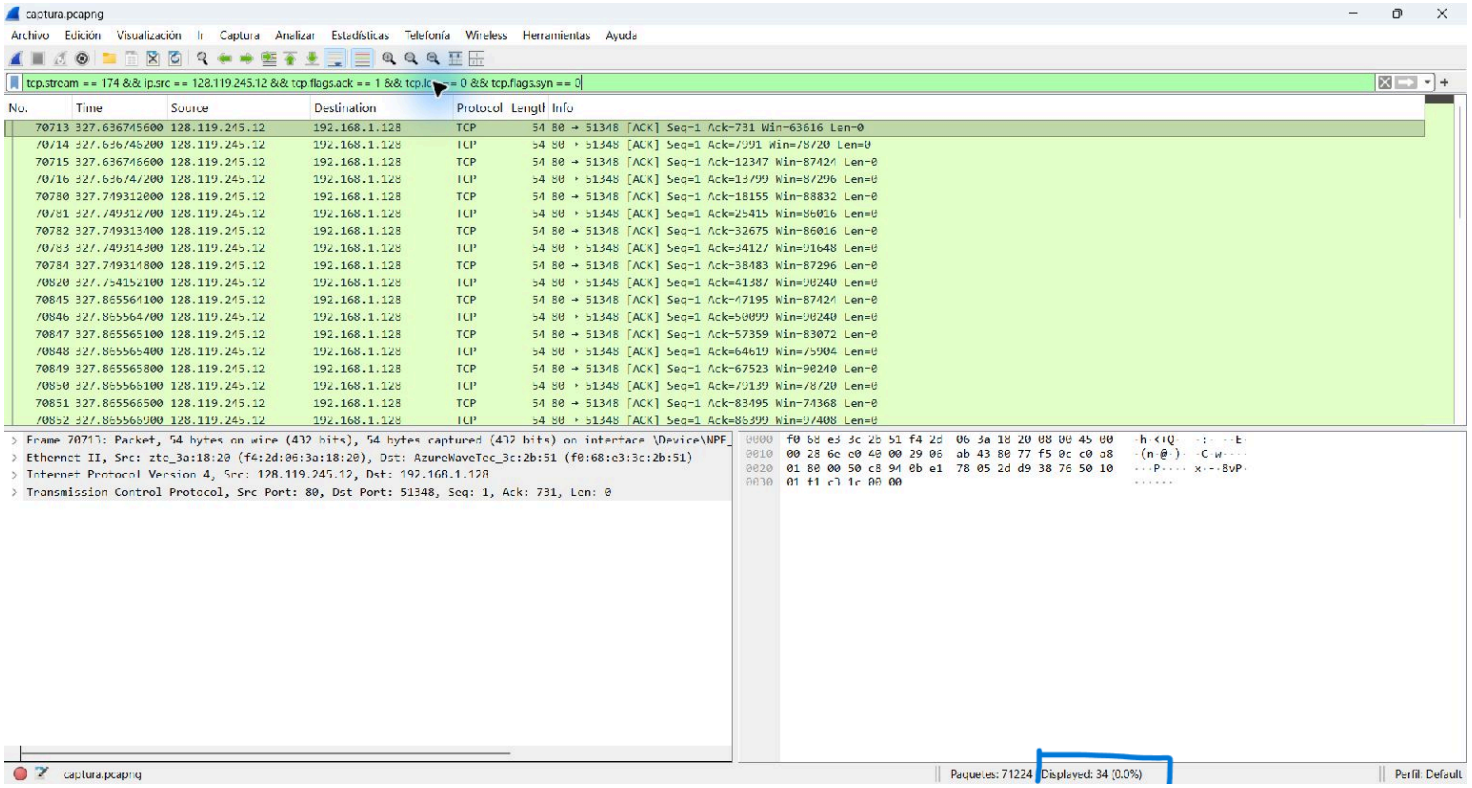


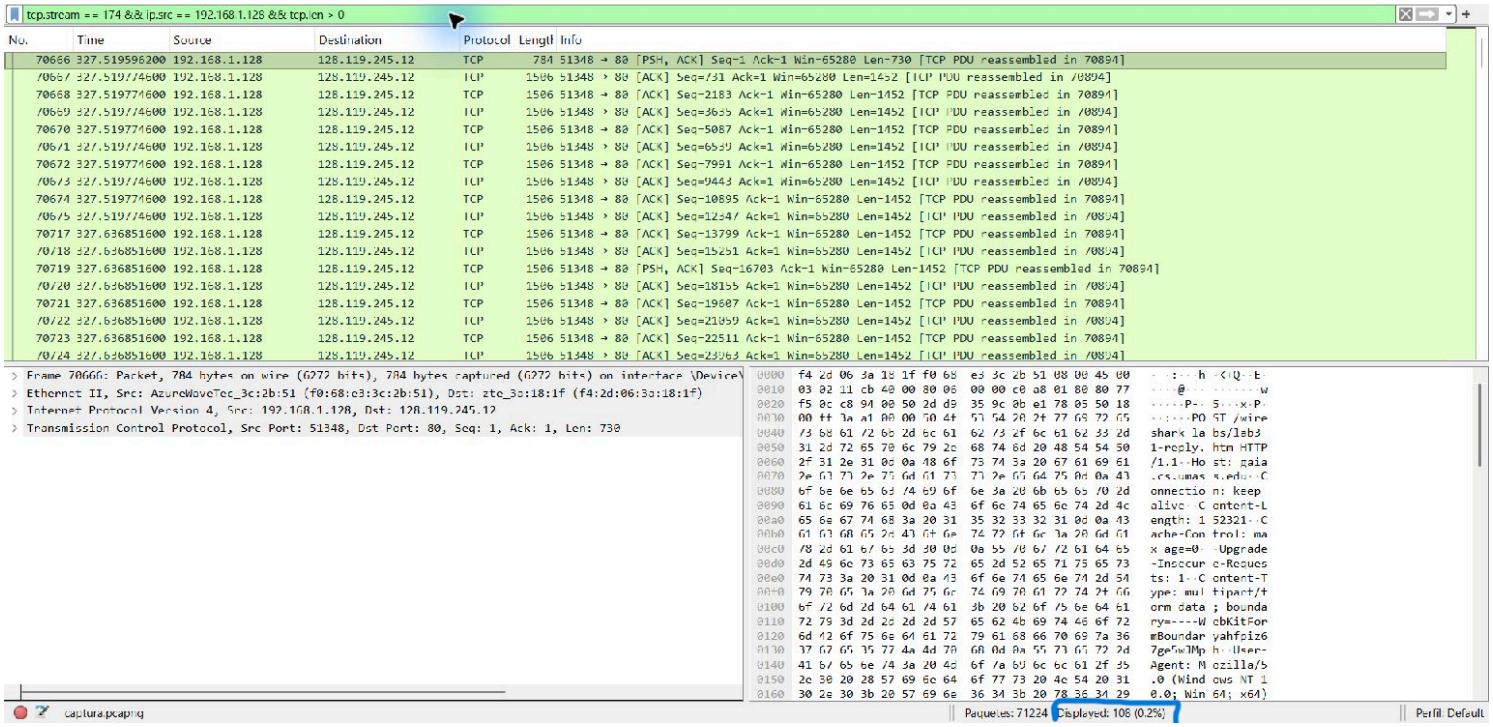

10. ¿En la captura realizada se han retransmitido segmentos? ¿Qué comprobaste en la traza para responder a esta pregunta?

| | |
|---|---|
| **Respuesta:** | **No hay ninguna. Los indicadores de retransmisión y retransmisión rápida están vacíos. Además, los números de secuencia del cliente solo crecen, no se repite ninguno y no hay ACK duplicados.** |



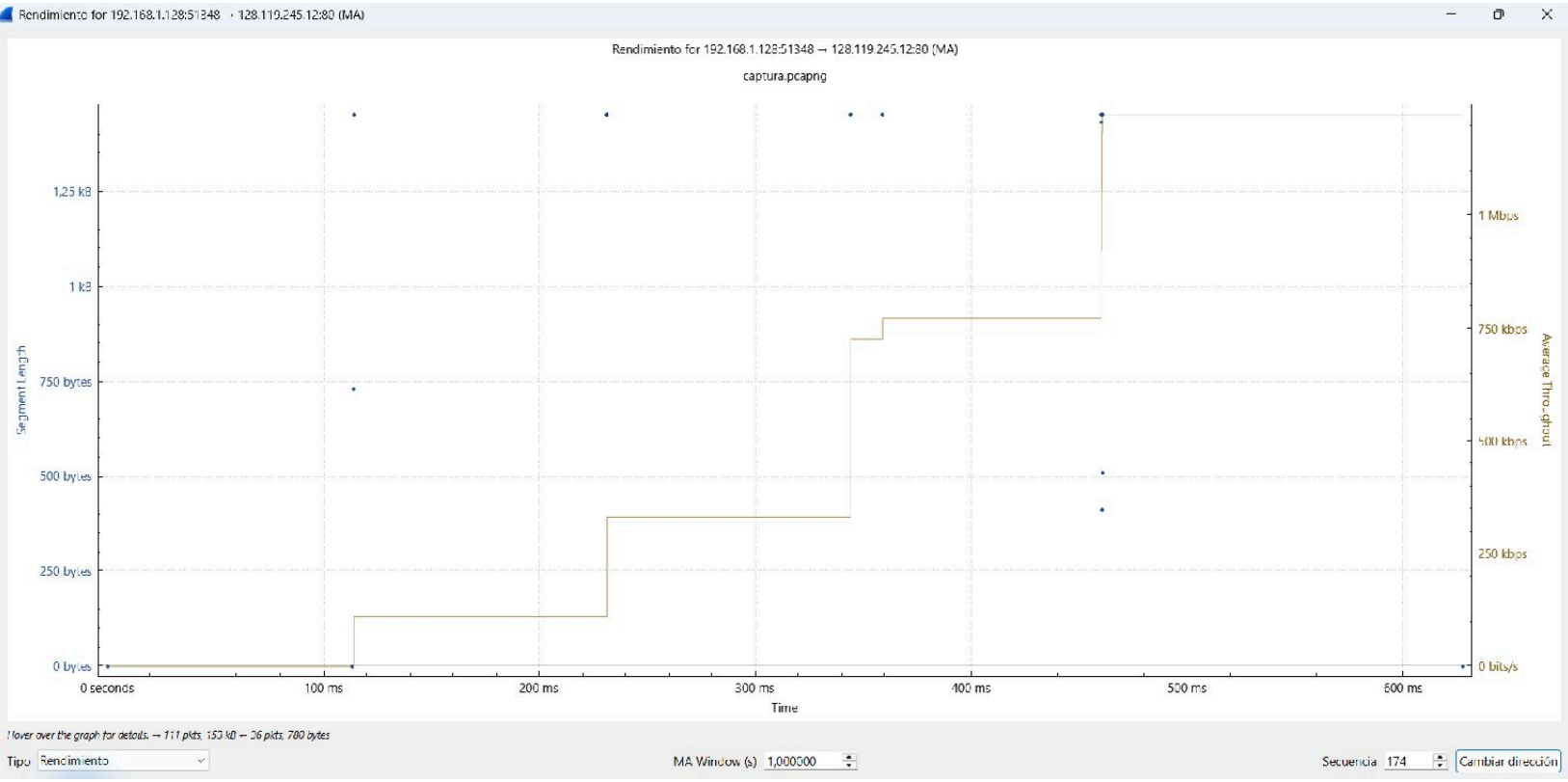

11. ¿Cuántos datos suele reconocer el receptor en un ACK? ¿Puedes identificar en la captura casos en que el receptor reconoce varios segmentos recibidos.


captura.pcapng
Archivo Edición Visualización Ir Captura Analizar Estadísticas Telefonía Wireless Herramientas Ayuda
tcp.stream == 174 && ip.src == 128.119.245.12 && tcp.flags.ack == 1 && tcp.len == 0 && tcp.flags.syn == 0
No. Time Source Destination Protocol Length Info
70713 327.636745600 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=731 Win=63616 Len=0
70714 327.636745200 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=7991 Win=78720 Len=0
70715 327.636746600 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=12347 Win=87424 Len=0
70716 327.636747200 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=13799 Win=87296 Len=0
70780 327.749312000 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=18155 Win=88832 Len=0
70781 327.749312700 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=25415 Win=86016 Len=0
70782 327.749313400 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=32675 Win=86016 Len=0
70783 327.749314000 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=34127 Win=91648 Len=0
70784 327.749314800 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=38483 Win=87296 Len=0
70820 327.754152100 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=41387 Win=90240 Len=0
70845 327.865564100 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=47195 Win=87424 Len=0
70846 327.865564700 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=50099 Win=90240 Len=0
70847 327.865565100 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=57359 Win=83072 Len=0
70848 327.865565400 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=64619 Win=79904 Len=0
70849 327.865565800 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=67523 Win=90240 Len=0
70850 327.865566100 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=73139 Win=78720 Len=0
70851 327.865566500 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=83495 Win=74368 Len=0
70852 327.865569000 128.119.245.12 192.168.1.128 TCP 54 80 → 51348 [ACK] Seq=1 Ack=86399 Win=97408 Len=0
Frame 70713: Packet, 54 bytes on wire (432 bits), 54 bytes captured (432 bits) on interface \Device\NPF_
Ethernet II, Src: zte_3a:18:20 (f4:2d:06:3a:18:20), Dst: AzureWaveTec_3c:2b:51 (f0:68:e3:3c:2b:51)
Internet Protocol Version 4, Src: 128.119.245.12, Dst: 192.168.1.128
Transmission Control Protocol, Src Port: 80, Dst Port: 51348, Seq: 1, Ack: 731, Len: 0
captura.pcapng
Paquetes: 71224 · Displayed: 34 (0.0%)
Perfil: Default


tcp.stream == 174 && ip.src == 192.168.1.128 && tcp.len > 0
No. Time Source Destination Protocol Length Info
70666 327.519596200 192.168.1.128 128.119.245.12 TCP 784 51348 → 80 [PSH, ACK] Seq=1 Ack=1 Win=65280 Len=730 [TCP PDU reassembled in 70894]
70667 327.519774600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=731 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70668 327.519774600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=2183 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70669 327.519774600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=3635 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70670 327.519774600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=5087 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70671 327.519774600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=6539 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70672 327.519774600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=7991 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70673 327.519774600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=9443 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70674 327.519774600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=10895 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70675 327.519774600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=12347 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70717 327.636851600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=13799 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70718 327.636851600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=15251 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70719 327.636851600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [PSH, ACK] Seq=16703 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70720 327.636851600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=18155 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70721 327.636851600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=19607 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70722 327.636851600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=21059 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70723 327.636851600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=22511 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
70724 327.636851600 192.168.1.128 128.119.245.12 TCP 1506 51348 → 80 [ACK] Seq=23963 Ack=1 Win=65280 Len=1452 [TCP PDU reassembled in 70894]
Frame 70666: Packet, 784 bytes on wire (6272 bits), 784 bytes captured (6272 bits) on interface \Device\
Ethernet II, Src: AzureWaveTec_3c:2b:51 (f0:68:e3:3c:2b:51), Dst: zte_3a:18:1f (f4:2d:06:3a:18:1f)
Internet Protocol Version 4, Src: 192.168.1.128, Dst: 128.119.245.12
Transmission Control Protocol, Src Port: 51348, Dst Port: 80, Seq: 1, Ack: 1, Len: 730
captura.pcapng
Paquetes: 71224 · Displayed: 108 (0.2%)
Perfil: Default

| | |
|---|---|
| **Respuesta:** | **El servidor envió 34 ACKs para 108 segmentos, unos 3 segmentos por ACK. Cada ACK confirma entre 1 y 8 segmentos. El caso típico son 4.356 bytes (3 segmentos) y la media es 4.501 bytes.** |

12. ¿Cuál es el throughput (bytes transferidos por unidad de tiempo) para la conexión TCP? Explica cómo calculaste este valor




(la imagen usa una media móvil de 1s)

| | |
|---|---|
| **Respuesta:** | **Hasta el último ACK: 153.051 B / 0,459701 s (de 327,519596 a 327,979297, ACK 70937) = 332,9 KB/s 2,66 Mbit/s.**<br><br>**Velocidad de envío: 153.051 B / 0,346665 s (hasta el último segmento enviado) 441,5 KB/s 3,53 Mbit/s.** |

## 2.4. Control de congestión en TCP

En este apartado trabajaremos estudiando los datos enviados por unidad de tiempo desde el cliente al servidor. Para ello, en lugar de realizar el cálculo sobre los datos en bruto en la ventana Wireshark, utilizaremos una de las utilidades TCP gráficas de Wireshark de representación de datos: *Time-Sequence-Graph (Stevens).*

- Selecciona un segmento TCP en la ventana de lista de paquetes capturados de Whireshark "*listing of captured-packets*". A continuación, selecciona el menú: *Statistics-> TCP Stream Graph-> Time-Sequence-Graph (Stevens).* La gráfica que aparecerá será semejante a la que te muestra la siguiente figura.

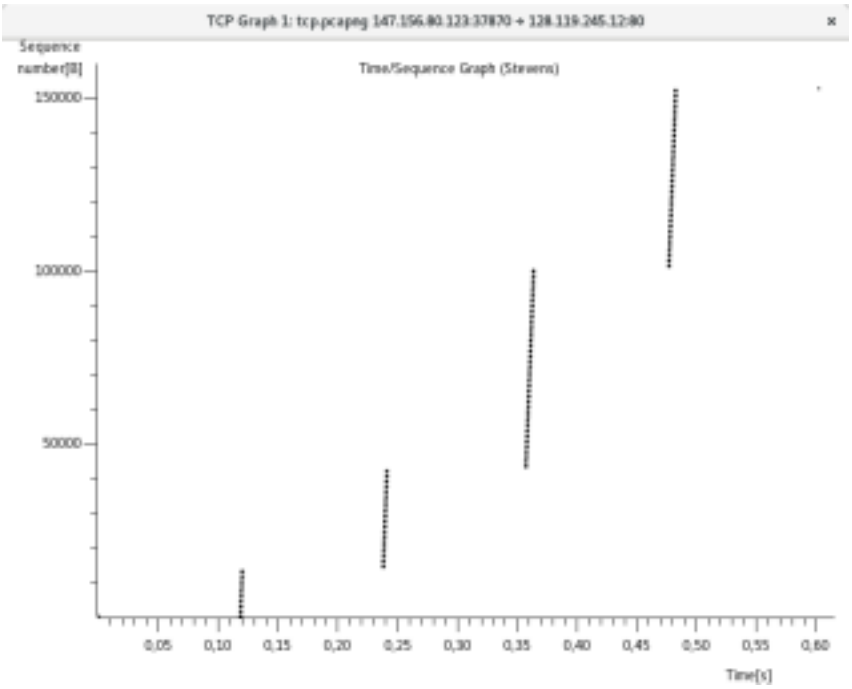



*Figura 7. Gráfica Time-Sequence-Graph (Stevens).*

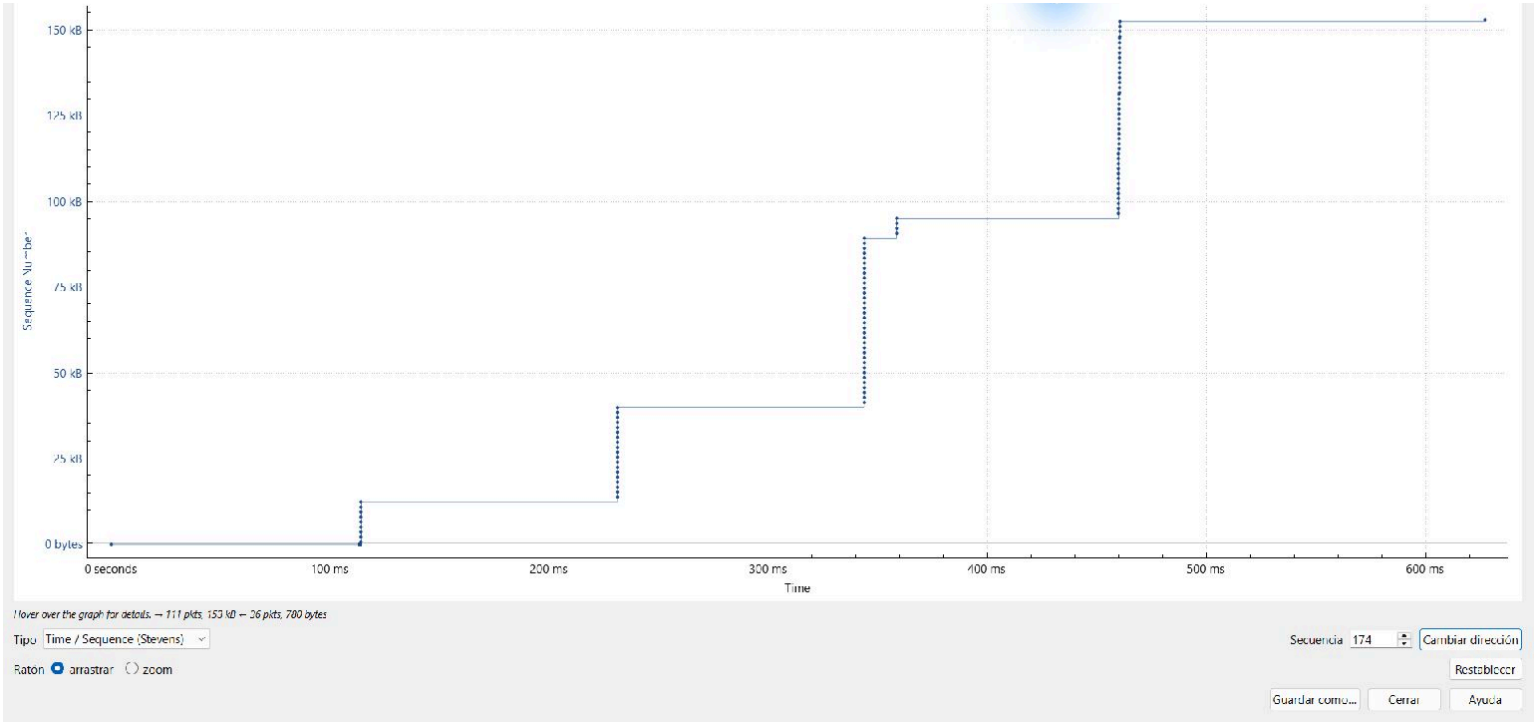
En la figura 7, cada punto representa un segmento TCP enviado, con su número de secuencia (eje de ordenadas) y el tiempo en que se envió (eje de abscisas). Ten en cuenta que un conjunto de puntos en la misma columna representan varios paquetes que fueron enviados al mismo tiempo (*back-to-back).*

Responde a las siguientes preguntas para los segmentos TCP de la captura que has realizado:

13. Utiliza la herramienta gráfica *Time-Sequence-Graph (Stevens)* para ver el número de secuencia en función del tiempo para los segmentos enviados desde el cliente al



servidor gaia.cs.umass.edu. Identifica dónde comienza y termina la fase de *Slowstart* de TCP, y dónde comienza el mecanismo de control de congestión para evitar la misma. Comenta en qué difiere el valor medido del comportamiento ideal de TCP estudiado en teoría y por qué. Responde a cada una de estas preguntas para la captura que has realizado.




| | |
|---|---|
| **Respuesta :** | **El Slow Start empieza en t = 114ms aproximado, con una ventana inicial de 10 segmentos. Cada RTT ( 115 ms aprox.) la ráfaga se duplica: x 2,0 de la 1 a la 2 y de 27,6 kB a 49,4 kB de la 2 a la 3. La ráfaga 4 es un resto, porque un ACK llegó 5 ms después del grupo principal.**<br>**Y para ver donde termina, tenemos que la ráfaga 5 solo tiene 56 KB porque el fichero se acaba, aunque la cwnd habría permitido unos 100 KB. No se llega a congestion avoidance, y esto es una inferencia a partir de los bytes en vuelo (Wireshark no muestra la cwnd).**<br>**Notamos las siguientes diferencias en comparacion a la teoría:**<br>● **La teoría parte de 1 MSS (1, 2, 4, 8…) y aquí son 10.**<br>● **Los ACKs acumulativos hacen que el crecimiento no sea exactamente ×2.**<br>● **No hubo pérdidas, así que no hay reducción de ventana.**<br>● **La transferencia dura solo unos 4 RTT.** |

### *3.- Estudio del protocolo UDP y DNS utilizando Wireshark.*

UDP es un protocolo del nivel de transporte basado en el intercambio de datagramas. Permite el envío de datagramas a través de la red sin que se haya establecido previamente una conexión, ya que el propio datagrama incorpora suficiente información de direccionamiento en su cabecera. Tampoco tiene confirmación ni control de flujo, por lo que los paquetes pueden adelantarse unos a otros; y tampoco se sabe si ha llegado correctamente, ya que no hay confirmación de entrega o recepción. Algunos protocolos de aplicación como DHCP, DNS y BOOTP funcionan sobre UDP. En esta práctica utilizaremos DNS como protocolo para analizar algunos campos de UDP.

El protocolo DNS (*Domain Name System)* traduce los nombres de host a direcciones IP, cumpliendo un papel crítico en la infraestructura de Internet. En este apartado, realizaremos un estudio en profundidad de DNS desde el punto de vista del cliente utilizando Whireshark. El papel del cliente en el DNS es relativamente sencillo: envía una consulta a su servidor DNS local, y recibe una respuesta. Sin embargo, antes de enviar la respuesta, se realizan comunicaciones transparentes para el cliente: como la comunicación jerárquica entre servidores DNS para resolver de manera recursiva o iterativa la solicitud DNS.
Desde el punto de vista del cliente DNS, sin embargo, el protocolo es muy simple: se formula una consulta al servidor DNS local y se recibe la respuesta desde el mismo.

Para realizar este apartado de la práctica es conveniente repasar los apuntes sobre los **servidores DNS locales**, el **almacenamiento en caché de DNS**, los **registros y mensajes DNS**, y el campo **TYPE** en el registro DNS.

Asimismo, necesitarás utilizar los comandos `nslookup` e `ip add`.

La sintaxis general del comando **nslookup** es:

```
nslookup –opcion1 –opcion2 host-a-encontrar servidor-dns
```

Lo podemos utilizar sin o con opciones, y especificando o sin especificar el servidor DNS.
Si no lo especificamos, responde a la petición el servidor DNS local.



Si queremos que responda a nuestra petición un servidor DNS diferente al local, lo especificaremos en `servidor-dns`.

Si necesitamos saber los nombres de los host de un determinado dominio, la opción que usaremos será: `–type=NS dominio`. En el caso de que quisiéramos conocer los nombres de los hosts asociados al dominio uv.es, la sintaxis sería:

```
nslookup –type=NS uv.es
```

Recuerda que **ip add** se utiliza para obtener toda la información TCP/IP de tu ordenador.

Averigua como borrar la caché local DNS de tu equipo.

| | |
|---|---|
| **Respuesta:** | **En ubuntu, el comando para borrar la cache del DNS local es:**<br>**nslookup –type=NS uv.es** |

### 3.1.- Captura y análisis de tráfico DNS con Wireshark.

En primer lugar realizaremos una captura de los paquetes DNS que se generan en una navegación web ordinaria. Vamos a capturar los paquetes DNS que son generados por la actividad de navegar en la web.

1. Vacía la memoria caché DNS del ordenador de prácticas.
2. Vacía la caché del navegador web que vayas a utilizar, en nuestro caso será Firefox (*Historial*  *Limpiar el historial reciente…, seleccionar todas las casillas y el rango temporal Todo*).
3. Abre Wireshark e inicia la captura de paquetes de Wireshark.
4. Con el navegador cuya caché has borrado, accede a la web: http://www.ietf.org
5. Detén la captura de paquetes.
6. Utiliza como filtro de visualización la dirección IP de tu ordenador, que habrás obtenido con `ip add`. El filtro a usar es "ip.addr == Direcc_IP ". Este filtro hace que Wireshark sólo **visualice** los paquetes cuyo origen o destino sea la dirección IP marcada (aunque captura todos los paquetes que el filtro de captura permita).

Responde a las siguientes preguntas con la captura de paquetes realizada, seleccionando la primera *Standard query* a ietf.org.

1. Localiza los mensajes de consulta y respuesta DNS. ¿Se envían sobre UDP o TCP?



| | |
|---|---|
| **Respuesta:** | **Se envía sobre UDP**<br>▸ User Datagram Protocol, Src Port: 34741, Dst Port: 53<br>Source Port: 34741<br>Destination Port: 53<br>Length: 49 |

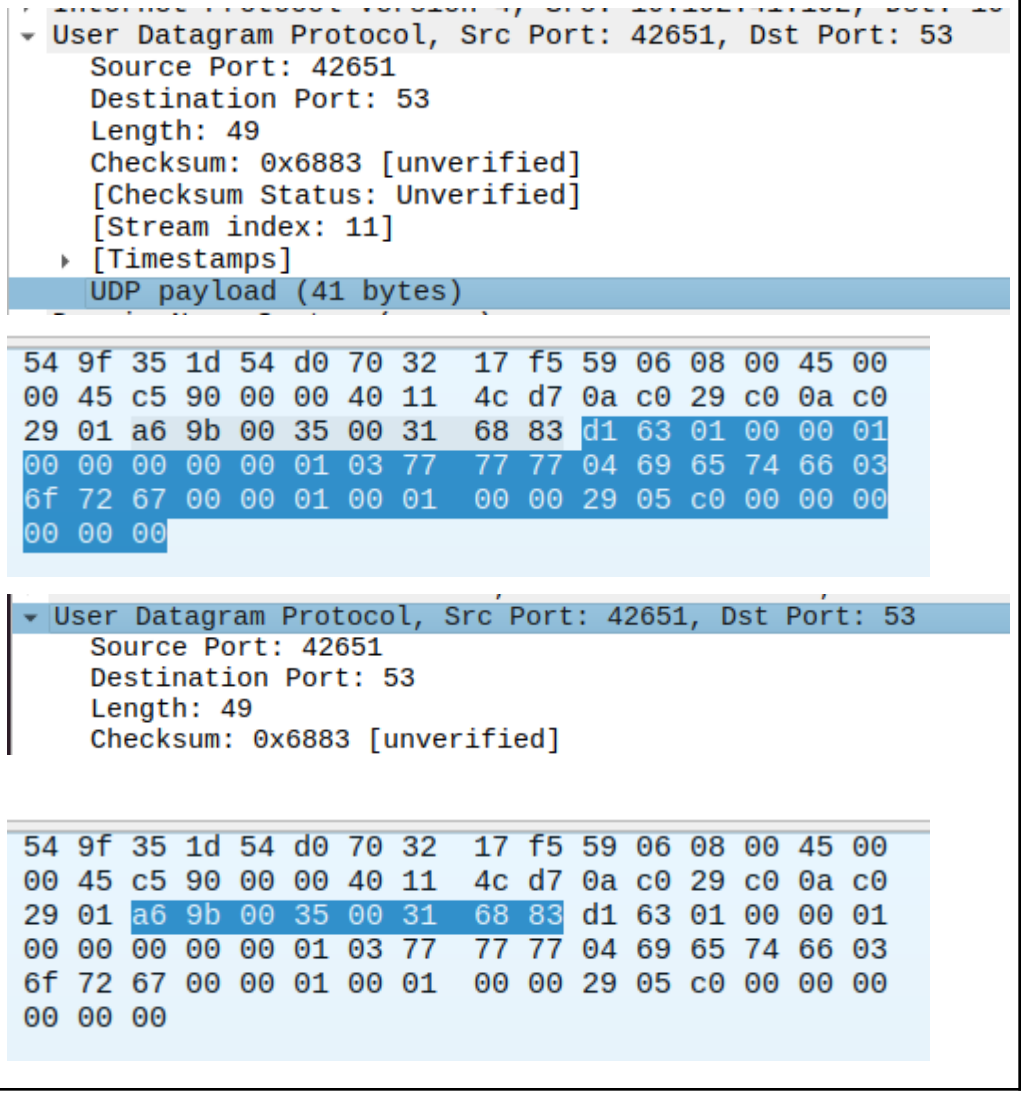
2. ¿Existe algún segmento de reconocimiento a nivel de transporte? ¿Por qué? Explica

con detalle las diferencias entre TCP y UDP.

| | |
|---|---|
| **Respuesta :** | **No, no existe. Esto se debe a la naturaleza de UDP que no es orientado a la conexión. A diferencia de TCP, UDP no tiene que reconocer la llegada de datagramas con un ACK ni nada parecido porque el UDP no existe el concepto de conexión ni está pensado para asegurar la llegada de datagramas al destino.** |

3. Identifica los campos del datagrama y su valor.

**Respuesta :**




4. ¿Qué longitud tiene el mensaje DNS? ¿Corresponde con el campo longitud del datagrama UDP? ¿Por qué? ¿Cuál es el límite práctico en IPv4 para la longitud de datos que se impone por el protocolo IP subyacente?

| | |
|---|---|
| **Respuesta :** | **No, la longitud del mensaje DNS no coincide con la longitud en la cabecera UDP porque esa longitud mide el tamaño del datagrama completo (cabecera UDP + payload) . El límite práctico es el MTU del enlace físico** |

5. ¿Que valores y/o campos se utiliza para calcular el campo checksum en UDP?

| | |
|---|---|
| **Respuesta :** | **Se utiliza la pseudo cabecera IP UDP** |

6. ¿Cuál es el puerto destino en el mensaje de consulta DNS? ¿Cuál es el puerto origen en el mensaje de respuesta de DNS?

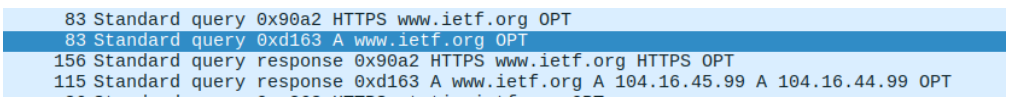


| | |
|---|---|
| **Respuesta :** | **El puerto destino es el 53 y el puerto de origen de la consulta también es el 53** |

7. ¿A qué dirección IP se envía el mensaje de consulta DNS? Utiliza **cat /run/systemd/resolve/resolv.conf** para determinar la dirección IP del servidor local de DNS. ¿Coinciden ambas direcciones IP?

| | |
|---|---|
| **Respuesta :** | **Se envia a 10.192.41.1 la cual coincide con la IP de mi servidor local DNS** |

8. Examina el mensaje de consulta DNS. ¿Qué "Tipo" de consulta DNS es? ¿El mensaje de consulta DNS contiene alguna "respuesta"?

| | |
|---|---|
| **Respuesta :** | **Las 2 primeras consultas consultan por registros HTTPS y A respectivamente y ambos tienen respuesta**<br>  |

9. Examina el mensaje de respuesta de DNS. ¿Cuántas "respuestas" provée? ¿Qué contiene cada una de estas respuestas?

| | |
|---|---|
| **Respuesta:** | **La respuesta de tipo A, nos da 2 respuestas que son 2 direcciones IP**<br>▾ Answers<br>▸ www.ietf.org: type A, class IN, addr 104.16.45.99<br>▸ www.ietf.org: type A, class IN, addr 104.16.44.99 |

10. Estudia ahora el paquete TCP SYN que envía tu ordenador con posterioridad. ¿La dirección IP destino del paquete SYN corresponde a alguna de las direcciones IP proporcionadas en el mensaje de respuesta de DNS?

| | |
|---|---|
| **Respuesta:** | Si, a la primera<br>104.16.45.99 TCP 74 41632 → 443 [SYN] |

11. La web a la que has accedido contiene imágenes. Antes de recuperar cada imagen, realiza tu host otras consultas DNS?



| | |
|---|---|
| **Respuesta:** | **Si realiza otras, sobre todo de tipo A**<br>DNS 83 Standard query 0x8da7 HTTPS www.ietf.org OPT<br>DNS 83 Standard query 0x5e08 AAAA www.ietf.org OPT<br>DNS 83 Standard query 0x358d A www.ietf.org OPT<br>DNS 244 Standard query response 0x8da7 HTTPS www.ietf.org HTTPS A 104.16.44.99 A 104.16.45.99 AAAA 2606:4700::6810:2c63 AAAA<br>DNS 115 Standard query response 0x358d A www.ietf.org A 104.16.45.99 A 104.16.44.99 OPT<br>DNS 139 Standard query response 0x5e08 AAAA www.ietf.org AAAA 2606:4700::6810:2d63 AAAA 2606:4700::6810:2c63 OPT<br>DNS 86 Standard query 0x40b7 HTTPS static.ietf.org OPT |

Capturaremos ahora el tráfico generado con las diferentes opciones del comando **nslookup**.

1. Inicia la captura de paquetes.
2. Realiza en el ordenador de prácticas la consulta `nslookup www.mit.edu`
3. Detén la captura de paquetes.

La captura obtenida se parecerá a la figura 8:

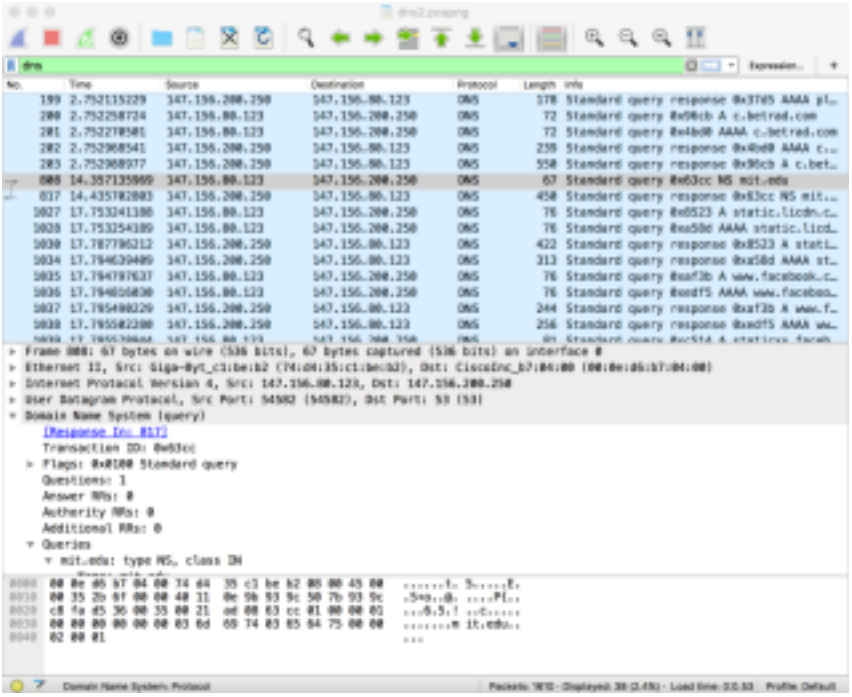


*Figura 8. Captura petición DNS a mit.edu*

En la imagen superior se observa que **nslookup** envía tres consultas DNS y recibe tres respuestas DNS. Para realizar este apartado, y a la hora de responder las siguientes preguntas, ignora las dos primeras parejas de consultas / respuestas, ya que son mensajes específicos de **nslookup** y normalmente no se generan por las aplicaciones estándar de Internet. Céntrate, por tanto, en los últimos mensajes de petición y respuesta.

12. ¿Cuál es el puerto destino en el mensaje de consulta DNS? ¿Cuál es el puerto origen en el mensaje de respuesta de DNS?



| | |
|---|---|
| **Respuesta :** | **El puerto de destino de la consulta es: 43**<br>▾ User Datagram Protocol, Src Port: 42880, Dst Port: 53<br>Source Port: 42880<br>Destination Port: 53<br>**El puerto de origen de la respuesta es:53**<br>▾ User Datagram Protocol, Src Port: 53, Dst Port: 42880<br>Source Port: 53<br>Destination Port: 42880 |

13. ¿A qué dirección IP se envía el mensaje de consulta DNS? ¿Es ésta la dirección IP de tu servidor DNS local por defecto?

| | |
|---|---|
| **Respuesta:** | **La direccion IP es 147.156.200.250 y coincide con la ip definida en /run/systemd/resolve/resolv.conf**<br><br>`Internet Protocol Version 4, Src: 10.20.57.1, Dst: 147.156.200.250` |

14. Examina el mensaje de consulta DNS. ¿Qué "Tipo" de consulta DNS es? ¿El mensaje de consulta DNS contiene alguna "respuesta"?

| | |
|---|---|
| **Respuesta:** | **La consulta es de tipo A, si la respuesta viene abajo**<br>`10.20.57.1 147.156.200.250 DNS 82 Standard query 0xa4ae A www.mit.edu OPT`<br>`147.156.200.250 10.20.57.1 DNS 171 Standard query response 0xa4ae A www.mit.e` |

15. Examina el mensaje de respuesta de DNS. ¿Cuántas "respuestas" provée? ¿Qué contiene cada una de estas respuestas?

| | |
|---|---|
| **Respuesta:** | **Nos da 3 respuestas, las primeras 2 son para resolver alias (CNAME) y la tercera devuelve la IP**<br>`Answers`<br>`www.mit.edu: type CNAME, class IN, cname www.mit.edu.edgekey.net`<br>`www.mit.edu.edgekey.net: type CNAME, class IN, cname e9566.dscb.akamaiedge.net`<br>`e9566.dscb.akamaiedge.net: type A, class IN, addr 23.37.166.128` |

### *4.- Estudio del protocolo de aplicación HTTP utilizando Wireshark.*

HTTP es el protocolo de comunicación que permite las transferencias de información en la World Wide Web sobre TCP/IP. Es un protocolo orientado a transacciones y sigue el esquema petición-respuesta entre un cliente y un servidor. El cliente realiza una petición enviando un mensaje, con cierto formato al servidor, El servidor le envía un mensaje de respuesta.

Finalmente, en este apartado, estudiaremos diferentes aspectos del protocolo HTTP: la interacción básica en la obtención de archivos cortos,-GET/respuesta-; los formatos de mensaje de HTTP; en la obtención archivos HTML de mayor tamaño; y la de archivos HTML con objetos incrustados. Para realizar el apartado conviene revisar la teoría sobre el funcionamiento de HTTP.



### 4.1.- La interacción básica HTTP GET / respuesta

Comenzaremos el estudio de HTTP con la descarga de un archivo HTML muy simple, corto y sin objetos incrustados. Sigue los siguientes pasos:

1. Inicia el navegador.
2. Inicia el analizador de paquetes de Wireshark, sin comenzar la captura. Utiliza como filtro *http* (en la ventana de especificación de filtro). De este modo, sólo se mostrarán en la captura los mensajes HTTP intercambiados.
3. Espera dos minutos (la justificación la veremos luego), y pon en marcha la captura de paquetes de Wireshark.
4. Introduce la siguiente ruta en el navegador:

16. htt sp://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file1.html 17. Se mostrará en el navegador un archivo HTML muy simple, de una sola línea. 5. Detén la captura de paquetes de Wireshark.

La captura obtenida con Wireshark será semejante a la mostrada en la figura 9.

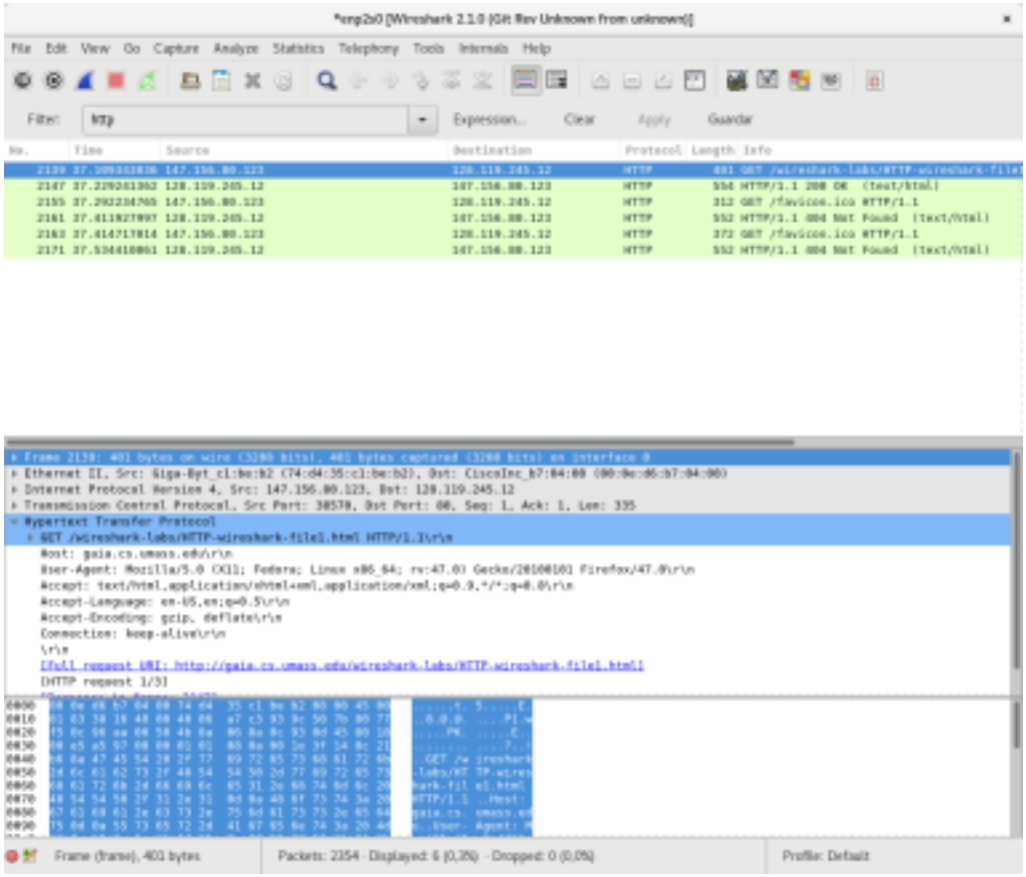


*Figura 9. Captura del protocolo HTTP con Wireshark.*

El ejemplo de la figura se observa en la ventana de listado de paquetes que se capturaron dos mensajes HTTP: el mensaje GET (desde el navegador al servidor web gaia.cs.umass.edu) y el mensaje de respuesta desde el servidor al navegador. La ventana de contenido de los paquetes muestra los detalles del mensaje seleccionado (en este caso, el



mensaje de confirmación HTTP, que se encuentra seleccionado destaca en la ventana de

listado de paquetes). Teniendo en cuenta que el mensaje HTTP se encapsuló en un segmento TCP, que posteriormente formó un paquete IP, que se transportó en una trama Ethernet, Wireshark muestra la información de todos los niveles. Asegúrate de ocultar la información de los niveles inferiores, y desplegar toda la información relativa a HTTP.

**Nota**: Ignora cualquier HTTP GET y la respuesta de *favicon.ico* Se trata de la solicitud automática desde el navegador al servidor de un icono para mostrar junto a la URL del servidor. En este apartado ignoraremos las referencias a este archivo, pues no son necesarias para el desarrollo del mismo.

Con la información obtenida en la captura de Wireshark responde a las siguientes preguntas.

1. ¿Qué versión de HTTP se ejecuta en el navegador? ¿Qué versión de HTTP se ejecuta en el servidor?

| | |
|---|---|
| **Respuesta:** | **El servidor ejecuta HTTP/1.1** |

2. ¿Qué idiomas (si los hay) indica el navegador que puede aceptar del servidor?

| | |
|---|---|
| **Respuesta:** | `Accept-Language: es-ES,es;q=0.9\r\n` |

3. ¿Cuál es la dirección IP de tu ordenador? ¿Y del servidor gaia.cs.umass.edu?

| | |
|---|---|
| **Respuesta:** | **La IP de mi ordenador es la 10.20.45.70, mientras que la del servidor es la 128.119.245.12** |

4. ¿Cuál es el código de estado que devuelve el servidor al navegador?

| | |
|---|---|
| **Respuesta:** | **Devuelve el código 200 OK** |

5. ¿Cuándo fue el archivo HTML actualizado por última vez en el servidor?

| | |
|---|---|
| **Respuesta:** | `Last-Modified: Tue, 28 Oct 2025 05:59:01 GMT\r\n` |

6. ¿Cuántos bytes de contenido se envían al navegador?

| | |
|---|---|
| **Respuesta:** | `Content-Length: 128\r\n` |

7. Observando los datos en bruto en la ventana de contenido de los paquetes, ¿localizas alguna cabecera de datos que no se muestre en la ventana de listado de paquetes? Si es así, nombra una y justifica tu respuesta.

| | |
|---|---|
| **Respuesta:** | **Si, observando los detalles identificamos cabeceras como Date, Server p ETag que no aparecen en la linea de información del listado de paquetes** |

En tu respuesta a la pregunta 5 de este punto, podría sorprender que el documento obtenido se actualizó por última vez justo un minuto antes de su descarga. Esto se debe a que (para este archivo en particular), el servidor establece como hora de última modificación la hora actual, y lo hace una vez por minuto. Por lo tanto, si se espera un minuto entre accesos, se actualizará la fecha de modificación del archivo, y por tanto el navegador descargará una "nueva" copia del documento.

### 4.2.- La interacción HTTP GET CONDICIONAL / respuesta

La mayoría de los navegadores funcionan realizando caché de objetos y por lo tanto realizan un GET condicional cada vez que recuperan un objeto HTTP. Antes de realizar los siguientes pasos, asegúrate de que la caché del navegador está vacía. (En *Firefox*, selecciona Herramientas> Limpiar el historial reciente y marca la casilla de la caché; en *Internet Explorer*, selecciona Herramientas> Opciones de Internet> Eliminar archivos): Realiza ahora el seguimiento y capura:

1. Inicia el navegador web, y vacía su caché.
2. Pon en marcha el analizador de paquetes Wireshark, y comienza la captura de paquetes.
3. Introduce la siguiente URL en el navegador:
4. http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file2.html
5. El navegador mostrará un archivo HTML de cinco líneas.
6. A continuación entra de nuevo en la misma URL (puedes pulsar el botón de

actualización del navegador).

7. Detén la captura de paquetes de Wireshark, y escribe *http* en la ventana especificación de filtro, para que en la captura sólo se muestren los mensajes HTTP intercambiados.

Responde las siguientes preguntas:

1. Inspecciona el contenido de la primera solicitud HTTP GET del navegador al servidor. ¿Existe una línea de "IF-MODIFIED-SINCE" en el HTTP GET?

| | |
|---|---|
| **Respuesta:** | **No, ya que al haber borrado la caché, el navegador solicitó el archivo desde cero sin ninguna referencia anterior** |

2. Inspecciona el contenido de la respuesta del servidor. ¿El servidor devolvió explícitamente el contenido del archivo? ¿Cómo puedes saberlo? Justifica tu respuesta.

| | |
|---|---|
| **Respuesta:** | **Si lo devolvió ya que el codigo de estado es 200 OK y la longitud del paquete (799 bytes) es lo suficientemente grande como para contener el archivo html** |

3. Ahora inspecciona el contenido de la segunda petición HTTP GET desde el navegador al servidor. ¿Existe una línea de "IF-MODIFIED-SINCE" en el HTTP GET? Si es así, ¿qué formato sigue y qué información contiene la cabecera "IF MODIFIED-SINCE:"?

| | |
|---|---|
| **Respuesta:** | **Existe y contiene una cabecera con la fecha y hora exacta en las que el archivo fue modificado por ultima vez**<br>`If-Modified-Since: Tue, 28 Oct 2025 05:59:01 GMT\r\n`<br>`\r\n` |

4. ¿Cuál es el código de estado HTTP y la frase devuelta por el servidor en respuesta a esta segunda petición HTTP GET? ¿El servidor devolvió explícitamente el contenido del archivo? ¿Cómo puedes saberlo? Justifica tu respuesta.

| | |
|---|---|
| **Respuesta:** | **El código de estado y la frase de vuelta son 304 Not Modified. El servidor no devolvió explicitamente el contenido del archivo y esto lo sabes porque el codigo 304 indica al navegador que use su caché ademas de que el tamaño de paquete (376) es un poco menos de la mitad que el original,** |

| | |
|---|---|
| | **confirmando así que biene vacio de datos** |

### 4.3. Obtención y descarga de documentos largos.



En los ejemplos utilizados en el apartado hasta el momento, los documentos descargados han sido archivos HTML simples y cortos. A continuación veamos lo que sucede cuando descargamos un archivo HTML largo. Sigue los siguientes pasos:

1. Inicia el navegador web, y asegúrate de vaciar la caché.
2. Pon en marcha el analizador de paquetes Wireshark, y comienza la captura.
3. Introduce la siguiente URL en el navegador:
4. http://gaia.cs.umass.edu/wireshark-labs/HTTP-wireshark-file3.html 5. El navegador mostrará la Declaración de Derechos de Estados Unidos. 6. Detén la captura de paquetes de Wireshark, y escribe *http* en la ventana
   especificación de filtro, para que en la captura sólo se muestren los mensajes HTTP intercambiados.

En la ventana de listado de paquetes, deberías ver el mensaje HTTP GET, seguido de una respuesta TCP multipaquete a la solicitud HTTP GET.
Recuerda que el mensaje de respuesta HTTP consiste en una línea de estado, seguido de líneas de cabecera, seguidas de una línea en blanco, y tras ellas por el cuerpo de la entidad. En el caso de nuestra HTTP GET, el cuerpo de la entidad en la respuesta es que todo el archivo HTML solicitado. En el caso que nos ocupa, el archivo HTML es bastante largo, y en 4500 bytes no caben en un único paquete TCP. De modo que un único mensaje de respuesta HTTP se fragmenta en el nivel de transporte por TCP, y cada fragmento se encapsula en un segmento TCP separado. En las últimas versiones de Wireshark, Wireshark indica cada segmento TCP como un paquete separado, y el hecho de que una única respuesta HTTP se fragmenta en múltiples paquetes TCP se indica con el mensaje "segmento TCP de una PDU reensamblada", (en inglés “TCP segment of a reassembled PDU”) en la columna Información de la pantalla Wireshark . Las versiones anteriores de Wireshark utilizan la palabra "Continuación", (Continuation) para indicar que el contenido de un mensaje HTTP se fragmentó en múltiples segmentos TCP.

Responde las siguientes preguntas:

1. ¿Cuántos mensajes de solicitud HTTP GET solicitud envió el navegador? En la captura realizada, ¿qué número de paquete contiene el mensaje GET para la solicitud de la Declaración de Derechos?

| | |
|---|---|
| **Respuesta:** | **El navegador envio un único mensaje de solicitud HTTP GET**<br>No. \| Time \| Source \| Destination \| Protocol \| Lengtl \| Info<br>731 3.847066 10.20.45.70 128.119.245.12 HTTP 554 GET |

2. ¿Qué número de paquete de la captura realizada contiene el código de estado y la frase asociada con la respuesta a la solicitud GET HTTP?



| | |
|---|---|
| **Respuesta:** | **El paquete que contiene el código de estado y la respuesta del servidor es el 731**<br>746 3.951939 128.119.245.12 10.20.45.70 HTTP 1216 HTTP/1.1 200 OK |

46. ¿Cuál es el código de estado y la frase en la respuesta?

| | |
|---|---|
| **Respuesta:** | **El codigo de estado y la frase devueltas son 200 OK** |

3. ¿Cuántos segmentos TCP de datos son necesarios para transportar la única respuesta HTTP que contiene el texto de la Declaración de Derechos?

| | |
|---|---|
| **Respuesta:** | **Son necesarios 4 segmentos TCP de datos**<br>[4 Reassembled TCP Segments (4864 bytes): #743(1238), #744(1238), #745(1238), #746(1150)]<br>[Frame: 743, payload: 0-1237 (1238 bytes)]<br>[Frame: 744, payload: 1238-2475 (1238 bytes)]<br>[Frame: 745, payload: 2476-3713 (1238 bytes)]<br>[Frame: 746, payload: 3714-4863 (1150 bytes)]<br>[Segment count: 4] |

## *5.- Bibliografía*

Adaptado de: Wireshark Labs Version: 7.0 © 2016 J.F. Kurose, K.W. Ross.

## Apéndice A: Filtros aplicados durante la captura por Wireshark.

*(**Nota**: estos comandos pueden sufrir modificaciones entre las diferentes versiones de Wireshark)*

En Wireshark es posible construir filtros que determinen si un determinado paquetes van a ser o no capturado. En caso de que no se utilice ningún filtro, todos los paquetes son capturados.

En Wireshark hay 2 tipos de filtros, los de captura que estamos comentando (*o capture filters*) y los de pantalla (*display filter*). Estos últimos son diferentes a los primeros, y sirven para una vez realizada la captura, filtrar lo que se presenta en pantalla. En este apéndice sólo analizamos los primeros, los de captura. Para utilizar los filtros de pantalla, es mejor utilizar la pestaña de Expressión para crear el Filtro en la pantalla mostrada en la Figura 4.

Los filtros se construyen mediante expresiones que consisten en una o más primitivas. Las primitivas, usualmente, consisten en un identificador (nombre o número), precedidas por uno o más calificadores. Existen tres tipos diferentes de calificadores:

- De tipo: Identifican un nombre o dirección, sus posibles valores son *host*, *net* y *port*. Algunos ejemplos son:

| **Filtro** | **Explicación** |
|---|---|
| `host glup.uv.es` | Paquetes cuyo origen o destino es el ordenador glup.uv.es. |

| | |
|---|---|
| `net 147.156.0.0/16` | Paquetes cuyo origen o destino es la red 147.156.0.0/16 |
| `port 20` | Paquetes cuyo origen o destino es el puerto 20. |

Si no existe ningún calificador de tipo, se asume que el tipo es *host*.

• De dirección: Identifican una dirección particular de transferencia, esto es, un origen o destino. Sus valores posibles son *src*, *dst*, *src or dst* y *src and dst*. Por ejemplo:

| **Filtro** | **Explicación** |
|---|---|
| `src glup.uv.es` | Paquetes cuyo origen es el ordenador glup.uv.es. |
| `dst net 147.156.0.0/16` | Paquetes destino es la red 147.156.0.0/16 |
| `src or dst port ftp-data` | Paquetes cuyo origen o destino es el puerto 20 (ftp-data). |

Si no se indica ningún calificador de dirección, se toma el calificador de dirección por defecto (*src or dst*).

• De protocolo: Identifican un protocolo particular. Sus valores posibles son *ether*, *fddi*, *tr*, *ip*, *ip6*, *arp*, *rarp*, *decnet*, *tcp* y *udp*. Por ejemplo[2]:

2 Si en algún momento se desea especificar una dirección ethernet en el filtro, está debe especificarse como XX:XX:XX:XX:XX:XX



| **Filtro** | **Explicación** |
|---|---|
| `ether src glup.uv.es` | Paquetes ethernet cuyo origen es el ordenador glup.uv.es. |
| `arp net 147.156` | Paquetes de protocolo ARP con origen o destino en la red 147.156. |
| `tcp port 21` | Paquetes cuyo protocolo es TCP y origen o destino en el puerto 21. |

Si no se especifica ningún protocolo, todos los protocolos que sean consistentes con la identificación de tipo son capturados.

Como puede verse por la explicación de los diferentes tipos de calificadores, siempre están presentes, aunque sea por defecto, los tres tipos de calificadores. Así, la expresión:

```
ip 147.156.222.65
```

Es equivalente a:

```
ip src or dst host 147.156.222.65
```

Pueden construirse filtros más complejos mediante la combinación de primitivas mediante la utilización de paréntesis y/o las palabras *and*, *or* y *not*, siendo la prioridad de *not* mayor que la de *and* y *or*, cuya prioridad entre si es igual. Así, por ejemplo:

```
host glup.uv.es and not port ftp
```

Indica que se capturen todos los paquetes cuyo origen o destino es glup.uv.es excepto aquellos cuyo puerto de origen o destino es el de ftp (puerto 21).

Un listado de las primitivas más utilizadas se encuentra en la siguiente tabla:

| **Primitiva** | **Descripción** |
|---|---|
| `dst host <ordenador>` | Verdad si el campo destino del paquete es el <*ordenador*>. |
| `src host <ordenador>` | Verdad si el campo origen del paquete es el <*ordenador*>. |
| `host <ordenador>` | Verdad si el campo origen o destino del paquete es el <*ordenador*>. |
| `Ether dst <ordenador>` | Verdad si la dirección ethernet de destino es el <*ordenador*>. |
| `Ether src <ordenador>` | Verdad si la dirección ethernet de origen es el <*ordenador*>. |
| `Ether host <ordenador>` | Verdad si la dirección ethernet de origen o destino es el <*ordenador*>. |
| `dst net <red>` | Verdad si la dirección de destino del paquete corresponde a una dirección de la <*red*>. |



| **Primitiva** | **Descripción** |
|---|---|
| `src net <red>` | Verdad si la dirección de origen del paquete corresponde a una dirección de la <*red*>. |
| `net <red>` | Verdad si la dirección de origen o destino del paquete corresponde a una dirección de la <*red*>. |

| | |
|---|---|
| `dst net <red> mask <mascara>` | Verdad si la dirección de destino del paquete corresponde a una dirección de la <*red*> de máscara <*mascara*>. |
| `src net <red> mask <mascara>` | Verdad si la dirección de origen del paquete corresponde a una dirección de la <*red*> de máscara <*mascara*>. |
| `net <red> mask <mascara>` | Verdad si las dirección de origen o destino del paquete corresponde a una dirección de la <*red*> de máscara <*mascara*>. |
| `dst net <red>/<longitud>` | Verdad si la dirección de destino del paquete corresponde a una dirección de la <*red*> cuya máscara se indica por <*longitud*>. |
| `src net <red>/<longitud>` | Verdad si la dirección de origen del paquete corresponde a una dirección de la <*red*> cuya máscara se indica por <*longitud*>. |
| `net <red>/<longitud>` | Verdad si las dirección de origen o destino del paquete corresponde a una dirección de la <*red*> cuya máscara se indica por <*longitud*>. |
| `dst port <puerto>` | Verdad si el paquete tiene como destino el puerto dado por <*puerto*>[3]. |
| `src port <puerto>` | Verdad si el paquete tiene como origen el puerto dado por <*puerto*>. |
| `port <puerto>` | Verdad si el paquete tiene como origen o destino el puerto dado por <*puerto*>. |
| `less <longitud>` | Verdad si el paquete tiene una longitud menor o igual que <*longitud*>. |
| `greater <longitud>` | Verdad si el paquete tiene una longitud mayor o igual que <*longitud*>. |

Además de las expresiones anteriores, existen expresiones del tipo:

`<expresión 1> <operador> <expresión 2>`

Donde <*operador*> es uno de los operadores de comparación <, >, <=, >=, = y *!=* (menor, mayor, menor o igual, mayor o igual, igual y distinto).

Por su parte, <*expresión 1*> y <*expresión 2*> son expresiones aritméticas compuestas por constantes enteras expresadas con la sintaxis de C/C++ y los operadores +, -, *, /, *&* y | (suma, resta, producto, división, and lógico y or lógico). Además, existe una

3 Esta expresión y las dos siguientes pueden ir precedidas de tcp ó udp, para indicar que solo se desea el puerto correspondiente al protocolo TCP ó UDP respectivamente.



expresión especial que permite acceder a unos datos concretos del paquete. Esta expresión que permite el acceso a los datos de un paquete tiene la sintaxis:

```
protocolo [ desplazamiento : tamaño]
```

Donde *protocolo* es uno de los protocolos validos (*ether*, *fddi*, *tr*, *ip*, *arp*, *rarp*, *tcp*, *udp*, *icmp* o *ip6*), *desplazamiento* es el desplazamiento, en bytes, desde el comienzo de los datos del protocolo especificado (el primer byte es el byte cero), y *tamaño* son los bytes a analizar, tomando por defecto el valor de 1 byte si no se especifica ningún valor. Por ejemplo:

```
ip[0 : 1] & 0x0F != 5
```

Indica todos los paquetes que contienen opciones IP (campo longitud de la cabecera con valor distinto de 5). Esta expresión también puede escribirse como:

```
ip[0] & 0x0F != 5
```

Teniendo en cuenta que si no se especifica el tamaño se toma por defecto 1. Por otra parte, el siguiente ejemplo:

```
ip[6 : 2] & 0x1FFF == 0
```

Indica solo datagramas no fragmentados o el primer fragmento de los datagramas fragmentados (campo desplazamiento del fragmento igual a cero).

**Otros comandos de IPv6:** Capture Filter (https://wiki.wireshark.org/IPv6) Captura de sólo tráfico IPv6:

```
ip6
```

Captura de tráfico IPv6 del host fe80::1:

```
host fe80::1
```

Captura tráfico de IPv6 en túnel IPv6-over-IPv4

```
ip proto 41
```

Captura de sólo tráfico IPv6 nativo:

```
ip6 and not ip proto 41
```

## Apéndice B: Filtros aplicados en la visualización de tráfico en Wireshark.

Los filtros de visualización (Display Filter) establecen los criterios de filtro sobre los paquetes capturados y que estamos visualizando en la pantalla principal de Wireshark.

Como puede verse en la figura 4, el área para escribir un filtro de captura es la ubicada debajo de la barra de herramientas. Wireshark nos proporciona una retroalimentación visual para realizar los filtros de visualización. Si la expresión de filtrado que escribimos es correcta, el área se colorea en verde mientras si la expresión es incorrecta, el área se colorea en rojo, y al tratar de aplicarlo obtendremos un mensaje de error.

Con los filtros de visualización veremos solo los paquetes que cumplan los requisitos indicados en el filtro. Si no establecemos ninguno, Wireshark mostrará todo el tráfico capturado y lo presentará en la pantalla principal.

Estos filtros son mucho más potentes que los de captura y mucho más fáciles de utilizar. La lista es inmensa como podemos ver en https://www.wireshark.org/docs/dfref/

Podemos combinar los filtros de la siguiente forma:

*Negación:* `!` ó `not`
*Concatenación*: `&&` ó `and`
*Alternancia*: `||` ó `or`

Los operadores que podemos utilizar para construir expresiones de filtro son los siguientes:

*Mayor que:* `>` ó `gt`
*Menor que:* `<` ó `lt`
*Mayor o igual que:* `>=` ó `ge`
*Menor o igual que:* `<=` ó `le`
*Igual que:* `==` ó `eq`
*No es igual a:* `!=` ó `ne`

*Contains*: busca la cadena *contains*

busca la cadena "google.es" en los paquetes tcp: `tcp contains "www.google.es"`

**Creación de filtros**

Los filtros de visualización ofrecen una gran cantidad de posibilidades a la hora de mostrar unos paquetes u otros. Por suerte, Wireshark posee un buscador para facilitarnos la tarea. Para familiarizarnos un poco con este tipo de filtros vamos a ver unos cuantos ejemplos.

**Filtrado por protocolo**

Para visualizar los paquetes **arp**
`arp`



Para visualizar paquetes **icmp** como el *ping*.
`icmp`

Para visualizar los paquetes **udp**
`udp`

Para visualizar los paquetes **tcp**
`tcp`

**Filtrar por direcciones ip**

Para visualizar todos los paquetes ip
`ip`

Para visualizar paquetes cuya ip de origen o destino es 192.168.100.1
`ip.addr == 192.168.100.1`

Para visualizar paquetes cuya ip de origen es 192.168.100.1
`ip.src == 192.168.100.1`

Para visualizar paquetes cuya ip de destino es 192.168.100.1
`ip.dest == 192.168.100.1`

**Podemos filtrar por puerto**

Para visualizar paquetes tcp cuyo puerto de origen o destino es el 80 `tcp.port == 80`

Para visualizar paquetes tcp cuyo puerto de origen es el 23
`tcp.srcport == 23`

Para visualizar paquetes tcp cuyo puerto de destino es el

23 `tcp.dstport == 23`

**Ejemplos de filtros http**

Para visualizar paquetes del protocolo de aplicación **http**
`http`

Para visualizar paquetes que tenga ha google como host
`http.host==”www.google.es”`

Para visualizar paquetes http que tenga una fecha concreta
`http.date==”Sun, 3 Mar 2013 15:32:33 GMT+1”`

Para visualizar paquetes http según su contenido
`http.content_type==”text/html”`



`http.content_type==”image/gif”`
`http.content_type==”text/css”`
`http.content_type==”image/png”`

`http.content_type==”image/jpeg”http.content_type==”applicati o n/json"`
`http.content_type==”application/zip"`
`http.content_type==”video/quicktime”`

Para visualizar paquetes http según con peticiones
`http.request.method==”GET”`
`http.request.method==”POST”`

Para visualizar paquetes http según el explorar
`http.user_agent contains “Android”`
`http.user_agent contains “Mozilla”`
`http.user_agent contains “Firefox”`
`http.user_agent contains “MSIE”`

Si queremos realizar algún filtro más detallado podemos utilizar el Display filter expression para ayudarnos. Ver la siguiente figura.


Wireshark · Display Filter Expression
Field Name
Relation
104apci · IEC 60870-5-104-Apci
104asdu · IEC 60870-5-104-Asdu
1722A · IEEE 1722a Protocol
29West · 29West Protocol
2dparityfec · Pro-MPEG Code of Practice...
3COMXNS · 3Com XNS Encapsulation
3GPP2 A11 · 3GPP2 A11
6LoWPAN · IPv6 over IEEE 802.15.4
802.11 Radio · 802.11 radio information
802.11 Radiotap · IEEE 802.11 Radiotap C...
802.11 RSNA EAPOL · IEEE 802.11 RSNA ...
802.3 Slow protocols · Slow Protocols
9P · Plan 9
A-bis OML · GSM A-bis OML
A21 · A21 Protocol
AAL1 · ATM AAL1
AAL3/4 · ATM AAL3/4
AARP · Appletalk Address Resolution Pro...
AASP · Aastra Signalling Protocol
ACAP · Application Configuration Access...
ACN · Architecture for Control Networks
ACP133 · ACP133 Attribute Syntaxes
ACR 122 · Advanced Card Systems ACR1...
ACSE · ISO 8650-1 OSI Association Cont...
ACtrace · AudioCodes Trunk Trace
ADB · Android Debug Bridge
ADB CS · Android Debug Bridge Client-S...
ADB Service · Android Debug Bridge Ser...
ADP · Aruba Discovery Protocol
ADwin · ADwin communication protocol
ADwin-Config · ADwin configuration prot...
is present
==
!=
>
<
>=
<=
contains
matches
Value
Predefined Values
Range (offset:length)
Search:
No display filter
A filter
Help
Cancel
OK